Compressible Flow Calculator

A PROJECT REPORT

submitted to

SRM INSTITUTE OF SCIENCE & TECHNOLOGY

Submitted by

RIMESH BHOWMICK-RA2411019010086

SUMNIMA RAI- RA2411019010108

AKELLA SREYESHA –RA2411019010145

RIDHI PAVAN KUMAR MITTAL - RA2411019010147

in partial fulfilment of the requirements for the degree of

BACHELOR OF TECHNOLOGY

IN

AEROSPACE ENGINEERING



DEPARTMENT OF AEROSPACE ENGINEERING

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY

KATTANKULATHUR, CHENGALPATTU DISTRICT

NOVEMBER 2024

Acknowledgement

I would like to thank Dr Raghavendra V (Assistant Professor, Dept of computing technologies) for guidance and feedback during this work, and Rimesh Bhowmick, Sumnima Rai, Akella Sreyesha, Ridhi Pavan Kumar Mittal for their support.

I am grateful to the authors of the classical literature on which every relation in this report rests, above all J. D. Anderson (*Modern Compressible Flow*), the Ames Research Staff (NACA Report 1135), H. W. Liepmann and A. Roshko, and A. H. Shapiro. I also thank the developers and maintainers of the open-source tools I used: Python, NumPy, SciPy, Matplotlib, pytest and Git/GitHub.

I used an AI assistant (Claude, Anthropic) to help review and extend the code, to run the tests and to draft this report. All numerical results in the report come from running the code, and the errors I found and corrected along the way are described openly in Chapter 4.

Abstract

Compressible-flow relations for a calorically perfect gas are the working tools of nozzle, inlet, wing and duct design, yet hand calculation from tables is slow and error-prone. In this report I describe how I reviewed, corrected and extended an open-source Python calculator that originally covered isentropic ratios, the normal shock and the Prandtl–Meyer function for Mach numbers near and above one.

My review of the original code exposed ten defects, among them an unprotected division by zero, no input validation (negative Mach numbers and $\gamma \leq 1$ returned plausible-looking nonsense), failure on array input, an inverse Prandtl–Meyer solver that failed at $\nu = 0$ and near the physical limit, and a validation routine that printed values without checking them.

I restructured the code into a reusable library (`compressible_flow.py`) and a command-line front end (`main.py`), and extended it with the area–Mach inverse, normal-shock inverses and the Pitot ratio, the full oblique-shock θ–β–M solution with weak and strong branches and a detachment limit, an expansion-fan solver, and Rayleigh and Fanno flow with choking detection.

I verified the result with 116 automated tests, built from published table values (NACA 1135, Anderson) and physical identities, which pass with 99 % statement coverage of the library. Round-trip accuracy of all inverse solvers is better than 3×10^{-13} (relative) for M between 1.001 and 30. During verification I found a subtle near-sonic tolerance bug in code I had just written, which collapsed any Mach number within 0.35 % of sonic to exactly 1; I fixed it and locked the fix in with regression tests. The result is a validated, documented tool suitable for teaching and preliminary design. The complete source code is given in Appendices D to G.

Keywords: compressible flow, gas dynamics, shock waves, Prandtl–Meyer expansion, Rayleigh flow, Fanno flow, numerical verification, Python.

Contents

1 Introduction **1**
1.1 Background 1
1.2 Problem Statement 1
1.3 Objectives 1
1.4 Scope and Limitations 1
1.5 Organisation of the Report 2
2 Theoretical Background **3**
2.1 Governing Assumptions 3
2.2 Isentropic Flow 3
2.3 Normal Shock 3
2.4 Oblique Shock 4
2.5 Prandtl–Meyer Expansion 5
2.6 Rayleigh Flow 5
2.7 Fanno Flow 6
2.8 Comparison of the Flow Models 6
3 Review of the Original Code **7**
4 Methodology and Process **8**
4.1 Steps 1 to 4: Acquisition, Reproduction, Design, Implementation 8
4.2 Step 5: Verification 8
4.3 Step 6: Investigating Discrepancies 9
4.4 Step 7: Accuracy Sweep and a Late Bug 9
5 Software Design and Function Reference **10**
5.1 Architecture 10
5.2 Function Reference 10
5.3 Command-Line Interface 11
5.4 Key Algorithmic Decisions 12
6 Results **13**
6.1 Test Results 13
6.2 Comparison with Published Tables 14
6.3 Numerical Accuracy of the Inverse Solvers 14
6.4 Worked Examples 15
6.5 Graphical Results 16
7 Discussion **21**
8 Conclusions and Future Work **22**
8.1 Conclusions 22
8.2 Future Work 22
References **23**
Appendix A Running the Code **24**
Appendix B Summary of Changes to the Repository **25**
Appendix C Validation Output **26**
Appendix D compressible_flow.py (library) **27**
Appendix E main.py (command-line interface) **34**
Appendix F test_compressible_flow.py (library tests) **39**
Appendix G test_main.py (command-line tests) **46**

List of Figures

Figure 1 Geometry of an attached oblique shock on a wedge: upstream streamlines (solid), shock wave at angle β (heavy line), and flow turned through θ and parallel to the wedge surface (dashed) 4

Figure 2 Prandtl–Meyer expansion fan at a convex corner 5

Figure 3 Development and verification workflow followed in this work 8

Figure 4 Software architecture: command-line front end, numerical library with six relation families, test suite and third-party dependencies 10

Figure 5 Number of automated tests per group (116 in total, all passing) 13

Figure 6 Left: error of the supersonic A/A^* inverse before and after the tolerance fix (log scale) 14

Figure 7 Isentropic stagnation-to-static ratios and area ratio A/A^* versus Mach number ($\gamma = 1.4$) 17

Figure 8 Normal-shock relations versus upstream Mach number M1 17

Figure 9 θ–β–M chart for $\gamma = 1.4$ 18

Figure 10 Prandtl–Meyer function ν(M) with its asymptotic limit νmax 19

Figure 11 Rayleigh flow ratios relative to the sonic state versus Mach number 19

Figure 12 Fanno flow ratios and friction parameter fL^*/D relative to the sonic state versus Mach number 20

List of Tables

Table 1 Organisation of the report2
Table 2 Summary of the flow models implemented in the software 6
Table 3 Defects found in the original code and their corrections 7
Table 4 Files in the delivered package 10
Table 5 Public functions of compressible_flow.py 11
Table 6 Test groups and what they verify 13
Table 7 Computed results against published reference values ($\gamma = 1.4$) 14
Table 8 Maximum relative round-trip error of each inverse in the final code 15
Table 9 Normal-shock results for $\gamma = 1.4$ 15
Table 10 Weak and strong oblique-shock solutions for $M1 = 3$, $\theta = 15°$ 15
Table 11 Sensitivity of shock and expansion limits to γ at $M1 = 2$ 16
Table 12 Changes made to the repository 25

Nomenclature

Symbol	Meaning	Unit
a	Speed of sound	m/s
A	Duct cross-sectional area	m^2
A^*	Area at the sonic condition ($M = 1$)	m^2
D	Duct hydraulic diameter	m
f	Darcy friction factor (4 × Fanning factor)	–
L, L^*	Duct length; length to reach $M = 1$	m
M	Mach number V/a	–
M_n	Mach number normal to a shock	–
p, p_0	Static and stagnation pressure	Pa
R	Specific gas constant	J/(kg·K)
s	Specific entropy	J/(kg·K)
T, T_0	Static and stagnation temperature	K
V	Flow velocity	m/s
β	Shock wave angle	deg
γ	Ratio of specific heats c_p/c_v	–
θ, θ_{max}	Flow deflection angle; maximum attached value	deg
μ	Mach angle $\arcsin(1/M)$	deg
ν	Prandtl–Meyer function	deg
ρ	Density	kg/m^3
Subscripts 1, 2	Upstream and downstream of a wave or duct section	–
Superscript $*$	Sonic ($M = 1$) reference state	–

1 Introduction

This chapter introduces the physical problem, the starting point of the work, its objectives and the organisation of the report.

1.1 Background

When a gas moves at speeds comparable to the local speed of sound, its density no longer stays constant, and the incompressible approximation fails. The Mach number $M = V/a$ governs the behaviour: for $M < 1$ disturbances propagate upstream and the flow adjusts smoothly, while for $M > 1$ information cannot travel upstream and the flow adjusts through discontinuities (shock waves) and smooth expansion fans.

The one-dimensional relations for a calorically perfect gas, which are analytic and compact, are the foundation of rocket and jet-engine nozzle design, supersonic inlet design, wind-tunnel operation, shock-tube analysis and the preliminary aerodynamics of high-speed vehicles [1, 2, 3].

1.2 Problem Statement

My starting point was a public repository, *Compressible_flow_calculator_Mach1*, which contains a single script `main.py`. It implements isentropic stagnation ratios, the area–Mach relation, normal-shock relations, the Prandtl–Meyer function and its numerical inverse, a validation printout, three plots and an interactive prompt.

It was a sound prototype, but it was not yet dependable: it could crash or silently return wrong answers on invalid input, could not be reused as a library, and lacked automated verification and several standard relations.

1.3 Objectives

The objectives of this work were to:

1. Review the code, reproduce and document its defects, and correct them.
2. Add the standard relations that a Mach-1-and-above calculator is expected to provide: oblique shocks, expansion fans, inverse relations, and the two classical duct flows with heat addition (Rayleigh) and friction (Fanno).
3. Verify the implementation against published tables and against physical identities, with an automated test suite whose results can be reproduced by anyone.
4. Document the theory, design, results and limitations in this report.

1.4 Scope and Limitations

I limit the work to steady, one-dimensional (or locally planar two-dimensional) flow of a calorically perfect gas, meaning constant specific heats and constant γ. I do not cover real-gas

effects (vibrational excitation, dissociation, ionisation), viscous boundary layers, shock–boundary-layer interaction, conical shocks or the method of characteristics.

1.5 Organisation of the Report

Table 1 summarises the content of each chapter.

Table 1: Organisation of the report.

Chapter	**Content**
2 Theoretical Background	Assumptions and governing relations for the six flow models
3 Review of the Original Code	Defects found in the starting code
4 Methodology and Process	Seven-step development and verification procedure
5 Software Design	Architecture, function reference, command-line interface
6 Results	Test results, accuracy, worked examples, graphical results
7 Discussion	Evidence, limits of verification, model limits
8 Conclusions and Future Work	Findings and recommended extensions

2 Theoretical Background

In this chapter I state the assumptions and the closed-form relations that I implemented in the software. The derivations follow Anderson [1], Shapiro [4] and Liepmann and Roshko [3].

2.1 Governing Assumptions

All relations assume an ideal gas, $p = \rho RT$, with constant specific-heat ratio $\gamma = c_p/c_v$ (1.4 for air at moderate temperatures). The speed of sound is $a = \sqrt{(\gamma RT)}$. Stagnation (total) conditions, subscript 0, are those obtained by decelerating the flow isentropically to rest. For adiabatic flow the stagnation temperature T_0 is conserved, while the stagnation pressure p_0 is reduced by any entropy-producing process.

2.2 Isentropic Flow

From the energy equation and the isentropic relations:

$$T_0/T = 1 + \tfrac{1}{2}(\gamma-1)M^2 \quad (2.1)$$

$$p_0/p = (T_0/T)^{\gamma/(\gamma-1)} \quad (2.2)$$

$$\rho_0/\rho = (T_0/T)^{1/(\gamma-1)} \quad (2.3)$$

$$A/A^* = (1/M)\cdot[(2/(\gamma+1))\cdot(1 + \tfrac{1}{2}(\gamma-1)M^2)]^{(\gamma+1)/(2(\gamma-1))} \quad (2.4)$$

A/A^* is the ratio of local duct area to the area at which $M = 1$ would occur. It has a minimum of 1 at $M = 1$, so every $A/A^* > 1$ corresponds to two Mach numbers, one subsonic and one supersonic. This double-valuedness is why the inverse function requires a *branch* argument.

2.3 Normal Shock

Conservation of mass, momentum and energy across a stationary shock, with $M_1 \geq 1$, gives:

$$M_2^2 = (1 + \tfrac{1}{2}(\gamma-1)M_1^2) / (\gamma M_1^2 - \tfrac{1}{2}(\gamma-1)) \quad (2.5)$$

$$p_2/p_1 = 1 + (2\gamma/(\gamma+1))(M_1^2 - 1) \quad (2.6)$$

$$\rho_2/\rho_1 = (\gamma+1)M_1^2 / ((\gamma-1)M_1^2 + 2) \quad (2.7)$$

$$T_2/T_1 = (p_2/p_1) / (\rho_2/\rho_1) \quad (2.8)$$

$$p_{02}/p_{01} = (\rho_2/\rho_1)^{\gamma/(\gamma-1)} \cdot[(\gamma+1)/(2\gamma M_1^2 - (\gamma-1))]^{1/(\gamma-1)} \quad (2.9)$$

$$(s_2 - s_1)/R = -\ln(p_{02}/p_{01}) \quad (2.10)$$

The downstream flow is always subsonic. As $M_1 \to \infty$, $M_2 \to \sqrt{((\gamma-1)/(2\gamma))} = 0.378$ and $\rho_2/\rho_1 \to (\gamma+1)/(\gamma-1) = 6$ for $\gamma = 1.4$, so density cannot rise without limit. The stagnation-pressure ratio is always below 1: the shock is irreversible.

A Pitot probe in supersonic flow sits behind a bow normal shock and reads the Rayleigh Pitot pressure [6]:

$$p_{02}/p_1 = [\tfrac{1}{2}(\gamma+1)M_1^2]^{\gamma/(\gamma-1)} / [2\gamma M_1^2/(\gamma+1) - (\gamma-1)/(\gamma+1)]^{1/(\gamma-1)} \quad (2.11)$$

2.4 Oblique Shock

A shock inclined at wave angle β to the upstream flow turns the flow by the deflection angle θ (Figure 1). Only the velocity component normal to the shock is processed; the tangential component is unchanged. With $M_{n1} = M_1 \sin\beta$, the property ratios are those of a normal shock evaluated at M_{n1}, and the downstream Mach number is $M_2 = M_{n2}/\sin(\beta - \theta)$.

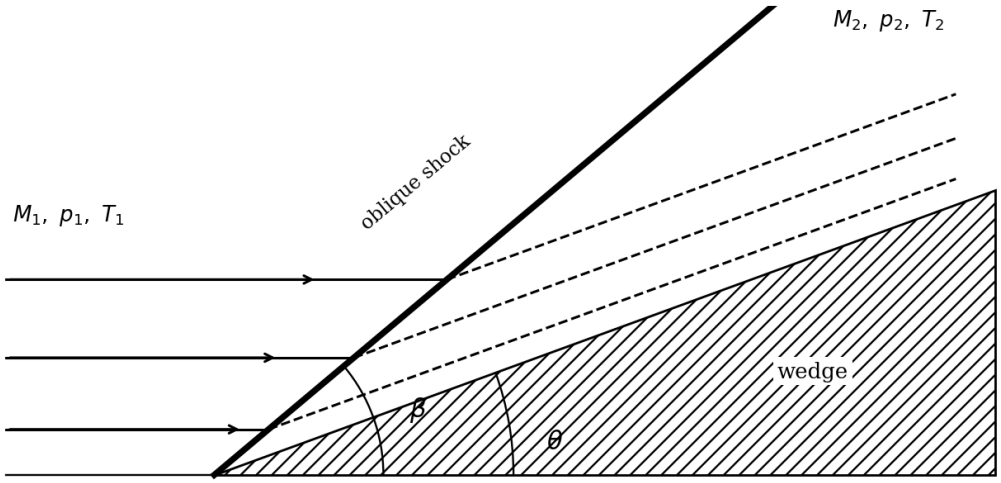


Figure 1: Geometry of an attached oblique shock on a wedge: upstream streamlines (solid), shock wave at angle β (heavy line), and flow turned through θ and parallel to the wedge surface (dashed).

The geometry links θ, β and M_1 through the θ–β–M relation:

$$\tan\theta = 2\cot\beta \cdot (M_1^2 \sin^2\beta - 1) / (M_1^2(\gamma + \cos 2\beta) + 2) \quad (2.12)$$

For a given M_1 there is a maximum deflection θ_{max}. For $\theta < \theta_{max}$ there are two solutions: the **weak** shock (smaller β, usually supersonic downstream, the one seen on wedges in free flight) and the **strong** shock (larger β, subsonic downstream). For $\theta > \theta_{max}$ no attached solution exists and the shock detaches and curves. The wave angle at θ_{max} is obtained in closed form:

$$\sin^2\beta_m = (1/(\gamma M_1^2))\cdot[\tfrac{1}{4}(\gamma+1)M_1^2 - 1 + \sqrt{((\gamma+1)(1 + \tfrac{1}{2}(\gamma-1)M_1^2 + (1/16)(\gamma+1)M_1^4))}] \quad (2.13)$$

The lower limit of β is the Mach angle $\mu = \arcsin(1/M_1)$, at which $\theta = 0$ and the shock degenerates into an infinitesimally weak Mach wave; the upper limit $\beta = 90°$ recovers the normal shock.

2.5 Prandtl–Meyer Expansion

When supersonic flow turns away from itself around a convex corner, it expands isentropically through a centred fan of Mach waves (Figure 2), first analysed by Meyer in 1908 [5].

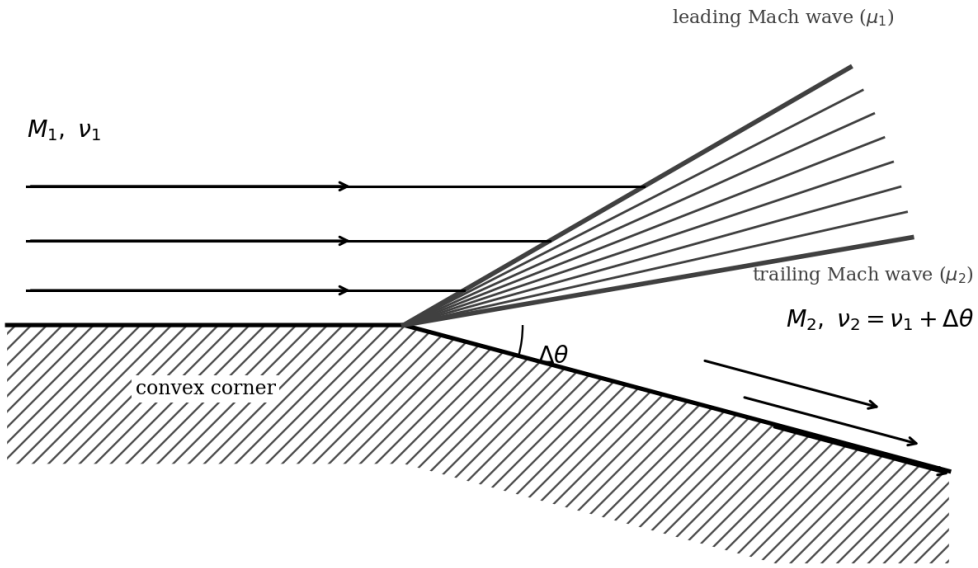


Figure 2: Prandtl–Meyer expansion fan at a convex corner. The fan is bounded by the leading Mach wave at μ_1 and the trailing Mach wave at μ_2; the flow turns through $\Delta\theta$ while accelerating from M_1 to M_2.

The Prandtl–Meyer function gives the angle through which sonic flow must turn to reach Mach number M:

$$\nu(M) = \sqrt{\frac{\gamma+1}{\gamma-1}}\cdot\arctan\sqrt{\frac{(\gamma-1)(M^2-1)}{\gamma+1}} - \arctan\sqrt{M^2-1} \tag{2.14}$$

$$\nu_{max} = \frac{\pi}{2}\left(\sqrt{\frac{\gamma+1}{\gamma-1}} - 1\right) \tag{2.15}$$

For $\gamma = 1.4$, $\nu_{max} = 130.45°$. For a turn $\Delta\theta$, $\nu_2 = \nu_1 + \Delta\theta$; M_2 follows from inverting ν, and the static ratios follow from the isentropic relations at M_1 and M_2 because p_0 and T_0 are conserved. The fan is bounded by Mach waves at $\mu_1 = \arcsin(1/M_1)$ and $\mu_2 = \arcsin(1/M_2)$. Because ν_{max} is finite, a gas cannot be expanded through more than $\nu_{max} - \nu_1$; beyond that a vacuum forms.

2.6 Rayleigh Flow

Rayleigh flow is steady, frictionless, constant-area flow with heat transfer [4]. Referred to the sonic state ($*$), with $d = 1 + \gamma M^2$:

$$\frac{T}{T^*} = \frac{(\gamma+1)^2 M^2}{d^2} \tag{2.16}$$

$$\frac{p}{p^*} = \frac{\gamma+1}{d} \tag{2.17}$$

$$\frac{\rho}{\rho^*} = \frac{d}{(\gamma+1)M^2}, \quad \frac{V}{V^*} = \frac{(\gamma+1)M^2}{d} \tag{2.18}$$

$$\frac{T_0}{T_0^*} = \frac{(\gamma+1)M^2(2 + (\gamma-1)M^2)}{d^2} \tag{2.19}$$

$$\frac{p_0}{p_0^*} = \frac{\gamma+1}{d}\cdot\left[\frac{2 + (\gamma-1)M^2}{\gamma+1}\right]^{\gamma/(\gamma-1)} \tag{2.20}$$

Heating drives both subsonic and supersonic flow toward M = 1 ($T_0/T_0{*} \to 1$). The maximum heat that can be added before the exit becomes sonic is fixed by the inlet Mach number; additional heating cannot be accepted without changing the upstream conditions, a condition called *thermal choking*. $T_0/T_0{*}$ is single-valued only on each branch: it has a lower limit $(\gamma^2-1)/\gamma^2 = 0.490$ as $M \to \infty$ on the supersonic branch.

2.7 Fanno Flow

Fanno flow is steady, adiabatic, constant-area flow with wall friction [4]. Referred to the sonic state:

$$T/T{*} = (\gamma+1) / (2 + (\gamma-1)M^2) \quad (2.21)$$

$$p/p{*} = (1/M)\cdot\sqrt{((\gamma+1) / (2 + (\gamma-1)M^2))} \quad (2.22)$$

$$\rho/\rho{*} = (1/M)\cdot\sqrt{((2 + (\gamma-1)M^2) / (\gamma+1))} \quad (2.23)$$

$$V/V{*} = M\cdot\sqrt{((\gamma+1) / (2 + (\gamma-1)M^2))} \quad (2.24)$$

$$p_0/p_0{*} = (1/M)\cdot[(2/(\gamma+1))(1 + \tfrac{1}{2}(\gamma-1)M^2)]^{(\gamma+1)/(2(\gamma-1))} \quad (2.25)$$

$$fL{*}/D = (1 - M^2)/(\gamma M^2) + ((\gamma+1)/(2\gamma))\cdot\ln[(\gamma+1)M^2 / (2 + (\gamma-1)M^2)] \quad (2.26)$$

Here f is the Darcy friction factor (four times the Fanning factor), and $L{*}$ is the duct length needed to bring the flow to M = 1. Friction drives both branches toward M = 1, stagnation pressure falls monotonically in the flow direction, and a duct longer than $L{*}$ is *choked*: the inlet conditions must change. On the supersonic branch $fL{*}/D$ has an upper limit of 0.8215 as $M \to \infty$ for $\gamma = 1.4$. Note that $p_0/p_0{*}$ in Fanno flow has the same mathematical form as $A/A{*}$ in isentropic flow, a useful cross-check used in the tests.

2.8 Comparison of the Flow Models

Table 2 compares the six models implemented.

Table 2: Summary of the flow models implemented in the software.

Model	Process	Conserved quantities	Limiting state
Isentropic	Area change, no losses	T_0, p_0, s	$A = A{*}$ at M = 1 (throat)
Normal shock	Irreversible compression	Mass, momentum, T_0	$M_2 < 1$, p_0 lost
Oblique shock	Compression, flow turned	Tangential velocity, T_0	θ_{max}: shock detaches
Prandtl–Meyer fan	Isentropic expansion	T_0, p_0, s	$\nu_{max} = 130.45°$: vacuum
Rayleigh	Heat addition, constant area	Mass, momentum	$T_0/T_0{*} = 1$: thermal choking
Fanno	Wall friction, adiabatic	Mass, energy (T_0)	$fL/D = fL{*}/D$: friction choking

3 Review of the Original Code

I read the original `main.py` (about 190 lines) in full and reproduced each defect below by running the code before I changed anything. The physics formulas that were already present (isentropic ratios, normal shock, Prandtl–Meyer) turned out to be correct, so I kept them. The defects concern robustness, structure and verification, and are listed in Table 3.

Table 3: Defects found in the original code and their corrections.

#	Defect in the original	Observed behaviour	Correction
D1	`area_mach_ratio(0)`	`ZeroDivisionError`	Mach number validated; clear `ValueError`
D2	No validation of M or γ	$M = -1$ returned $T_0/T = 1.2$; $\gamma = 0.9$ returned $T_0/T = 0.8$, both meaningless	Central `_check_mach` and `_check_gamma` guards
D3	`normal_shock` on a NumPy array	`ValueError`: truth value of an array is ambiguous	Vectorised, array-safe checks
D4	`mach_from_pm_angle` used a fixed bracket [1.0001, 50]	Failed for $\nu = 0$ and for $\nu = 130°$ ($\nu(50)$ is only 124.7°, so large turns were unreachable)	Analytic ν_{max}; expanding bracket; $\nu = 0$ returns 1
D5	`run_validation` only printed values	A regression would pass silently	PASS/FAIL checks with tolerances; non-zero exit code
D6	Normal-shock plot started at $M_1 = 2$	Range 1–2 omitted; p_{02}/p_{01} not plotted	Range 1–5; p_{02}/p_{01} added
D7	`plt.show()` after every plot	Blocked batch runs and servers	Plots saved by default; `--show` is opt-in
D8	Interactive prompt	Crashed on $M \leq 0$, $\gamma \leq 1$, or end of input	Errors reported, menu continues; end of input handled
D9	`area_mach_ratio` defined but unused	Dead code	Used in plots, summary and the new inverse
D10	Library and interface in one script	Not importable without the I/O code	Split into module and command-line interface

4 Methodology and Process

I followed the seven-step procedure shown in Figure 3. Steps 5 to 7 form a verification loop: whenever a check failed or I found a new defect, I corrected the code and re-ran the whole suite.

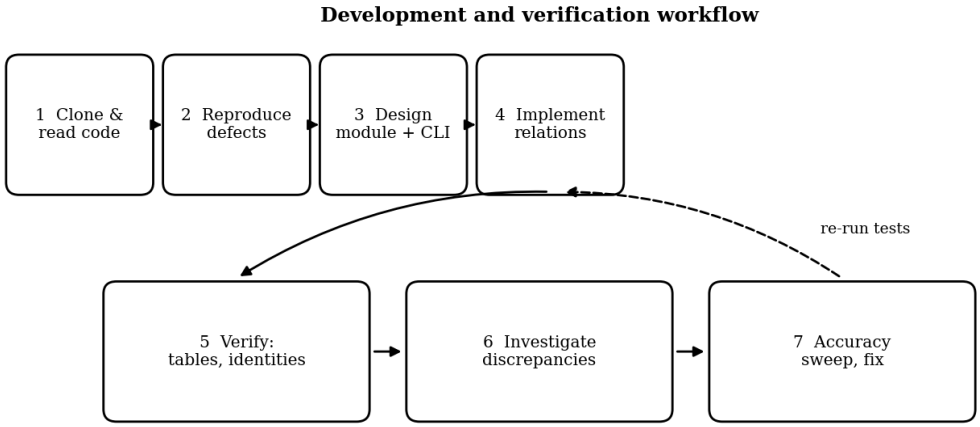


Figure 3: Development and verification workflow followed in this work.

4.1 Steps 1 to 4: Acquisition, Reproduction, Design, Implementation

Step 1: Acquisition and reading. I cloned the repository and read `main.py` in full. I also noted the three plots that were already committed.

Step 2: Defect reproduction. I triggered each suspected defect (D1 to D4 in Table 3) with a short script to confirm that it was real rather than assumed.

Step 3: Design. I moved the numerical core into an importable module without input/output side effects and left interaction to a separate `main.py`. Results are returned as named tuples, so tuple unpacking still works, and angles are in degrees at the interface. Every function validates its inputs and raises `ValueError` with an informative message instead of returning NaN or crashing.

Step 4: Implementation. Forward relations are closed-form and vectorised. Inverse relations are solved with Brent's method [9] (`scipy.optimize.brentq` [11]), a bracketing root-finder that is guaranteed to converge once a sign change is bracketed. Each inverse takes a `supersonic` flag to select the branch, because A/A^*, T_0/T_0^* and fL^*/D are all double-valued about $M = 1$. Brackets are chosen from physics (for example, the weak oblique-shock root lies in $[\mu, \beta_m]$ and the strong root in $[\beta_m, 90°]$) or expanded geometrically until a sign change appears.

4.2 Step 5: Verification

I wrote three kinds of tests: (a) comparison with published table values at several Mach numbers; (b) physical identities that do not depend on any table; (c) error handling and edge cases. Section 6.1 lists the results.

4.3 Step 6: Investigating Discrepancies

My first test run gave 90 passes and 2 failures. I traced both by hand instead of loosening the tolerances:

- ρ_0/ρ at $M = 3$: the test expected 13.125 and the code gave 13.119. Direct calculation, $2.8^{2.5} = 7.84 \times 1.6733 = 13.119$, shows that the code was correct and that I had mis-entered the reference.
- p_2/p_1 for $M_1 = 2$, $\theta = 20°$: the test expected 2.82 and the code gave 2.843. The textbook figure of 2.82 results from rounding M_{n1} to 1.60; with the exact $M_{n1} = 1.607$ the normal-shock relation gives 2.843. The test now documents this and also asserts agreement with the exact relation.

4.4 Step 7: Accuracy Sweep and a Late Bug

I then ran a systematic round-trip sweep (forward function, then inverse, over many Mach numbers), which revealed a relative error of order 10^{-3} in the supersonic area–Mach inverse, far above solver precision (Figure 6, left panel).

The cause was in code I had just written: `np.isclose(A, 1.0, atol=1e-12)` silently applies NumPy's default relative tolerance of 10^{-5}. Near the sonic point $A/A* - 1$ grows only as $(M - 1)^2$, so a 10^{-5} tolerance on A corresponds to roughly 3.5×10^{-3} in M. The function therefore returned exactly 1.0 for every Mach number below 1.00347 (an error of 10^{-3} at $M = 1.001$, which is what a coarse sweep first showed, and 3.5×10^{-3} in the worst case). The same flaw was present in the Rayleigh inverse, where the cut-off was $M = 1.0038$.

I replaced both with pure absolute tolerances and added a dedicated regression class of 11 tests. After the fix the worst error fell below 3×10^{-13} for $M \geq 1.001$. This episode shows why round-trip sweeps complement table spot-checks: table values at $M = 0.5$, 2 and 3 could never have exposed the fault.

Finally, I ran the full suite again from a clean copy of the files to confirm that it does not depend on leftover state.

5 Software Design and Function Reference

5.1 Architecture

The software is split into a numerical library with no input/output, a command-line front end, and a test suite (Figure 4). The library depends only on NumPy and SciPy; Matplotlib is used by the front end alone, so the library can be imported in headless or minimal environments.

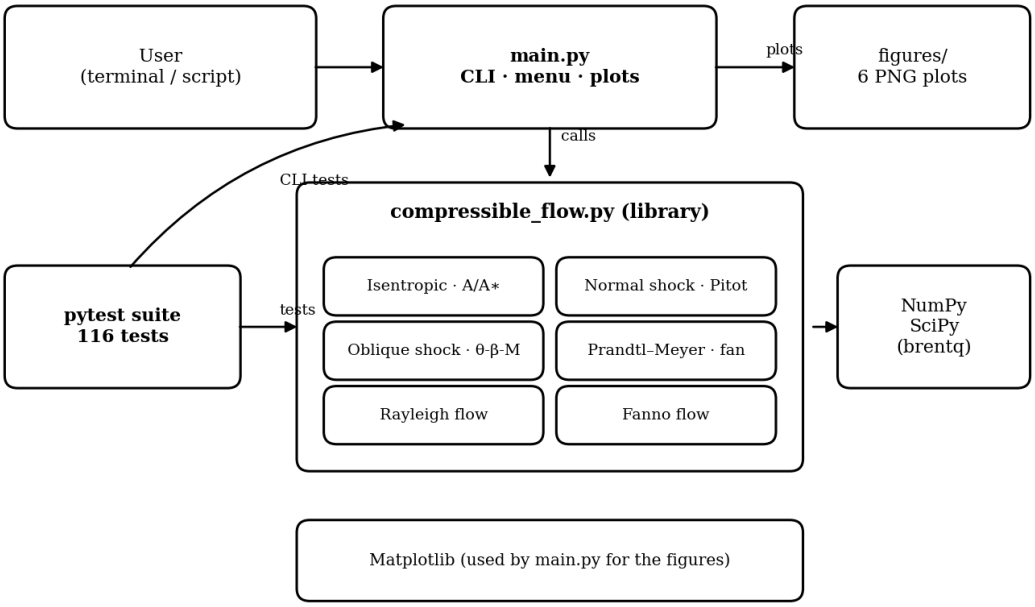


Figure 4: Software architecture: command-line front end, numerical library with six relation families, test suite and third-party dependencies.

Table 4 lists the files delivered.

Table 4: Files in the delivered package.

File	Lines	Role
`compressible_flow.py`	488	Numerical library; no input/output
`main.py`	318	Command-line interface: validation, six plots, one-shot summary, interactive menu
`test_compressible_flow.py`	439	108 library tests
`test_main.py`	54	8 command-line smoke tests
`README.md`, `requirements.txt`, `.gitignore`		Documentation and environment

5.2 Function Reference

Table 5 documents every public function of `compressible_flow.py`. All angles are in degrees.

Table 5: Public functions of `compressible_flow.py`.

Function	Purpose	Returns / raises
`isentropic_ratios(M, γ)`	T_0/T, p_0/p, ρ_0/ρ	`IsentropicState`; $M \geq 0$
`area_mach_ratio(M, γ)`	A/A^*	float or array; $M > 0$
`mach_from_area_ratio(A/A*, supersonic, γ)`	Inverse of A/A^* on the chosen branch	M; $A/A^* \geq 1$
`normal_shock(M₁, γ)`	M_2, p_2/p_1, ρ_2/ρ_1, T_2/T_1, p_{02}/p_{01}	`NormalShockState`; $M_1 \geq 1$
`shock_entropy_rise(M₁, γ)`	$(s_2 - s_1)/R$	float
`pitot_pressure_ratio(M₁, γ)`	p_{02}/p_1 (Rayleigh Pitot)	float
`mach1_from_pressure_ratio(p₂/p₁, γ)`	Inverse: M_1 from the static-pressure jump	M_1
`mach1_from_downstream_mach(M₂, γ)`	Inverse: M_1 from post-shock Mach number	M_1; M_2 in [0.378, 1]
`mach_angle_deg(M)`	$\mu = \arcsin(1/M)$	degrees
`theta_from_beta(M₁, β, γ)`	θ–β–M relation	θ in degrees
`max_deflection_angle(M₁, γ)`	θ_{max} and its β	(θ_{max}, β)
`oblique_shock(M₁, θ or β, weak, γ)`	Full oblique-shock state, weak or strong branch	`ObliqueShockState`; error if detached
`prandtl_meyer(M, γ)`	ν(M)	degrees
`prandtl_meyer_max(γ)`	ν_{max}	degrees
`mach_from_pm_angle(ν, γ)`	Inverse of ν(M)	M; $\nu < \nu_{max}$
`prandtl_meyer_expansion(M₁, Δθ, γ)`	Fan solver: M_2, ratios, Mach-wave angles	`ExpansionState`
`rayleigh_ratios(M, γ)`	Six Rayleigh ratios to the sonic state	`RayleighState`
`mach_from_rayleigh_T0_ratio(r, supersonic, γ)`	Inverse of T_0/T_0^*	M
`rayleigh_exit(M₁, T₀₂/T₀₁, γ)`	Exit Mach number after heating or cooling	M; error if thermally choked
`fanno_parameter(M, γ)`	fL^*/D	float
`fanno_ratios(M, γ)`	Five Fanno ratios plus fL^*/D	`FannoState`
`mach_from_fanno_parameter(p, supersonic, γ)`	Inverse of fL^*/D	M
`fanno_exit(M₁, fL/D, γ)`	Exit Mach number of a duct of given length	M; error if choked

5.3 Command-Line Interface

Running `python main.py` performs the self-validation (16 checks), generates the six figures into `figures/`, and opens a menu with seven actions: full summary at one Mach number; oblique shock; expansion fan; Mach number from A/A^*; Rayleigh exit; Fanno exit; and change of γ. The flags `--validate`, `--plots`, `--mach M`, `--gamma G`, `--no-plots`, `--no-interactive` and `--show` support scripting. Invalid physical input produces a one-line message and the menu continues.

5.4 Key Algorithmic Decisions

- **Branch selection** is explicit (`supersonic=True/False`, `weak=True/False`) rather than guessed.
- **Bracketing** guarantees convergence; no function uses an initial-guess method that could diverge.
- **Absolute tolerances only** are used at sonic points, for the reason found in Section 4.4.
- **Fail loudly:** choked ducts, detached shocks and unreachable ν raise `ValueError` rather than return an extrapolated or NaN value.

6 Results

This chapter reports the outcome of the verification, the accuracy of the inverse solvers, worked engineering examples and the graphical output of the program.

6.1 Test Results

All **116 tests pass**, both in the working directory and in a clean copy. Figure 5 and Table 6 show how the tests are distributed.

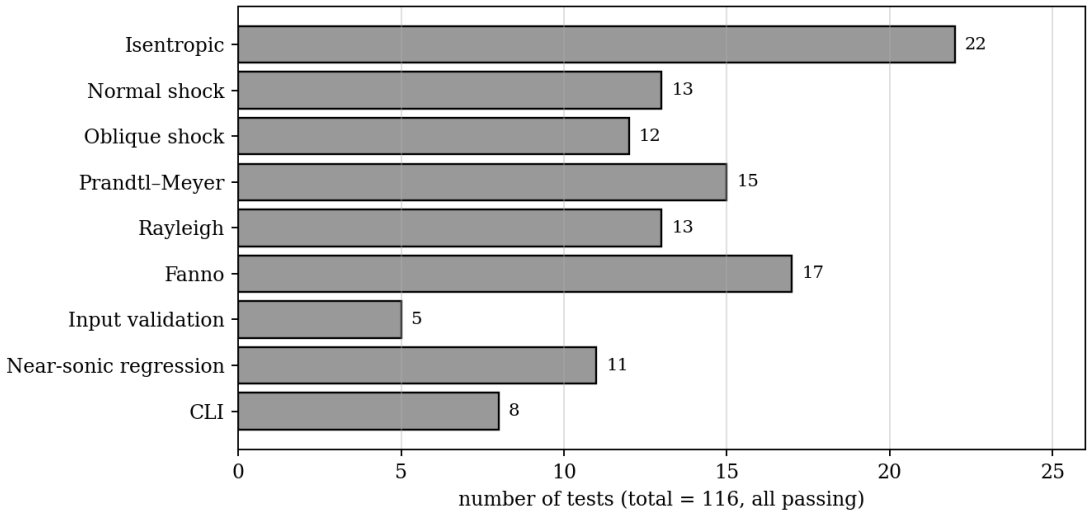


Figure 5: Number of automated tests per group (116 in total, all passing).

Table 6: Test groups and what they verify.

Test group	Tests	Content
Isentropic	22	Table values at M = 0.5, 1, 2, 3; ideal-gas identity; inverse round trips on both branches; invalid input
Normal shock	13	Table values at M_1 = 2, 3, 5; $M_2 < 1$; p_0 loss; density limit 6; Pitot cross-check; inverses; arrays
Oblique shock	12	Table and textbook cases; θ_{max}; detachment error; weak versus strong; $\beta = 90°$ equals normal shock; tangential-velocity conservation
Prandtl–Meyer	15	Table values; limit 130.45°; monotonicity; round trips; fan solver and isentropic consistency
Rayleigh	13	Table values; heating and cooling trends; thermal choking; inverse branches
Fanno	17	Table values; $p_0/p_0* = A/A*$; $p = \rho RT$ identity; supersonic limit; choking; leg additivity
Input validation	5	Remaining error paths
Near-sonic regression	11	Guards against the defect of Section 4.4
Command line (`test_main.py`)	8	Validation, summary, bad arguments, plots written, menu errors, end of input
Total	**116**	

Coverage: `compressible_flow.py` 99 % (281 of 283 statements; the two uncovered are the bracket-failure guard in the Prandtl–Meyer inverse and an exact-sonic early return in the Rayleigh inverse); `main.py` 85 %; combined 93 %. The built-in validation reports 16 of 16 checks passed (Appendix C).

6.2 Comparison with Published Tables

Table 7 compares computed values with the published tables for $\gamma = 1.4$.

Table 7: Computed results against published reference values ($\gamma = 1.4$).

Case	Quantity	Computed	Published
Isentropic M = 2	p_0/p	7.8244	7.8244
Isentropic M = 2	A/A^*	1.6875	1.6875
Normal shock $M_1 = 2$	M_2 / p_2/p_1 / p_{02}/p_{01}	0.5774 / 4.5000 / 0.7209	0.5774 / 4.5000 / 0.7209
Normal shock $M_1 = 5$	p_2/p_1 / T_2/T_1	29.000 / 5.8000	29.00 / 5.800
Pitot $M_1 = 2$	p_{02}/p_1	5.6404	5.6404
Oblique $M_1 = 2$, $\theta = 10°$	β / M_2 / p_2/p_1	39.31° / 1.641 / 1.707	39.31° / 1.641 / 1.707
Oblique $M_1 = 2$	θ_{max}	22.97°	22.97°
Prandtl–Meyer M = 2	ν	26.380°	26.380°
Rayleigh M = 2	T_0/T_0^* / p/p^*	0.7934 / 0.3636	0.7934 / 0.3636
Fanno M = 2	fL^*/D / p/p^*	0.3050 / 0.4082	0.3050 / 0.4082

6.3 Numerical Accuracy of the Inverse Solvers

I fed each inverse the output of its forward function and compared the recovered Mach number with the original. Figure 6 shows the result: the left panel isolates the defect of Section 4.4 and its removal, and the right panel shows the final round-trip error of the four Mach-number inverses, all at the level of floating-point rounding. Table 8 gives the numbers.

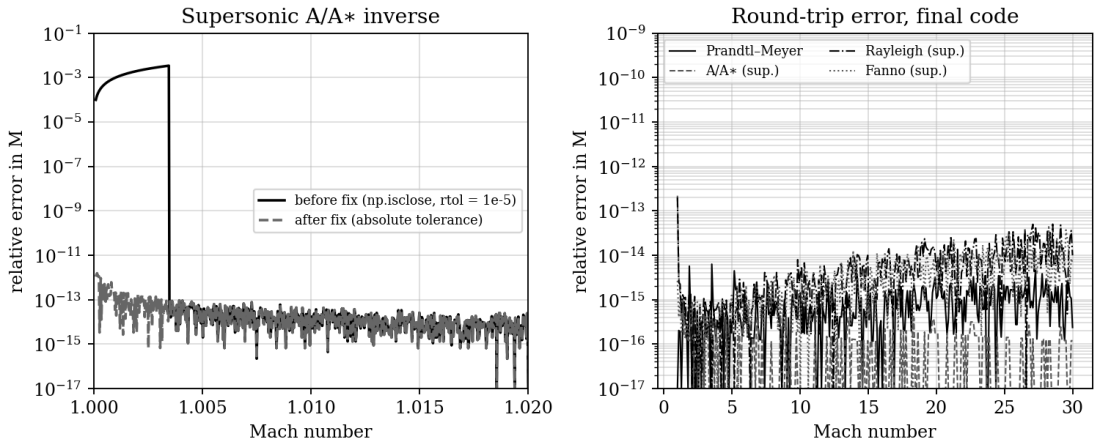


Figure 6: Left: error of the supersonic A/A^* inverse before and after the tolerance fix (log scale). Right: round-trip error of the four inverse solvers in the final code.

Table 8: Maximum relative round-trip error of each inverse in the final code.

Inverse	Range tested	Max relative error
Prandtl–Meyer $\nu \to M$	M = 1.001–30	6×10^{-15}
$A/A^* \to M$ (supersonic)	M = 1.001–30 (2000 points)	2×10^{-13} (before the fix: 3.5×10^{-3} worst case)
$A/A^* \to M$ (subsonic)	M = 0.05–0.99	2×10^{-14}
Rayleigh $T_0/T_0^* \to M$	subsonic and supersonic	2×10^{-13}
Fanno $fL^*/D \to M$	subsonic and supersonic	5×10^{-14}
Oblique $\theta \to \beta$ (weak and strong)	M_1 = 1.2–10, 300 cases	1×10^{-13} degrees (absolute)

Very close to M = 1 the problem itself is ill-conditioned ($A/A^* - 1$ varies as $(M-1)^2$), so the error grows to about 1.5×10^{-12} at M = 1.0001. Typical run time is 0.18 ms per oblique-shock solution and 0.13 ms per Prandtl–Meyer inversion, so the library is fast enough for parameter sweeps and optimisation loops.

6.4 Worked Examples

(a) Nozzle with an area ratio of 2. The subsonic root is M = 0.306 (p_0/p = 1.067); the supersonic root is M = 2.197 (p_0/p = 10.65). Which one occurs depends on the back pressure, a classic design question.

(b) Normal shock table. Table 9 lists the downstream state for five upstream Mach numbers.

Table 9: Normal-shock results for $\gamma = 1.4$.

M_1	M_2	p_2/p_1	ρ_2/ρ_1	T_2/T_1	p_{02}/p_{01}	$(s_2 - s_1)/R$
1.5	0.7011	2.458	1.862	1.320	0.9298	0.073
2	0.5774	4.500	2.667	1.688	0.7209	0.327
3	0.4752	10.33	3.857	2.679	0.3283	1.114
5	0.4152	29.00	5.000	5.800	0.0617	2.785
10	0.3876	116.5	5.714	20.39	0.0030	5.794

Entropy production grows rapidly: at M_1 = 10 only 0.3 % of the stagnation pressure survives the shock.

(c) Wedge in Mach 3 flow, θ = 15°. θ_{max} at M_1 = 3 is 34.07°, so an attached shock exists. Table 10 compares the two solutions.

Table 10: Weak and strong oblique-shock solutions for M_1 = 3, θ = 15°.

Branch	β	M_2	p_2/p_1	T_2/T_1	p_{02}/p_{01}
Weak	32.24°	2.255	2.822	1.388	0.895
Strong	84.42°	0.509	10.23	2.662	0.332

The weak shock is far more efficient, retaining 89.5 % of stagnation pressure against 33 % for the strong shock, which is why supersonic inlets are designed for weak-shock operation.

(d) Maximum deflection angle. θ_{max} rises with Mach number: 3.94° ($M_1 = 1.2$), 12.11° (1.5), 22.97° (2), 34.07° (3), 41.12° (5), 44.43° (10), approaching 45.58° as $M_1 \to \infty$.

(e) Expansion at $M_1 = 2$, turn 10°. ν rises from 26.38° to 36.38°, giving $M_2 = 2.385$, $p_2/p_1 = 0.548$, $T_2/T_1 = 0.842$ and $\rho_2/\rho_1 = 0.651$, with the fan spanning Mach waves from 30.00° to 24.79°. Compression by the same 10° through an oblique shock raised the pressure by a factor of 1.707, a useful contrast in magnitudes.

(f) Heating a subsonic duct, $M_1 = 0.3$. The thermal-choking limit is $T_{02}/T_{01} = 2.883$. Doubling the stagnation temperature (ratio 2.0) accelerates the flow to $M_2 = 0.502$ while the static pressure falls by 17 % ($p_2/p_1 = 0.833$) and the stagnation pressure by 7 % ($p_{02}/p_{01} = 0.929$).

(g) Friction. A subsonic duct with $M_1 = 0.5$ can support at most $fL^*/D = 1.069$. With $fL/D = 0.5$ the exit Mach number is 0.581 and $p_{02}/p_{01} = 0.904$. A supersonic duct with $M_1 = 3$ has $fL^*/D = 0.522$, so $fL/D = 0.2$ decelerates the flow to $M_2 = 2.059$.

(h) Sensitivity to γ. Table 11 shows the effect of γ at $M_1 = 2$. A lower γ (more complex molecules) lowers shock efficiency and allows larger turning angles; a monatomic gas ($\gamma \approx 1.67$) does the reverse.

Table 11: Sensitivity of shock and expansion limits to γ at $M_1 = 2$.

γ	p_{02}/p_{01}	ν_{max}	θ_{max}
1.1	0.648	322.4°	29.25°
1.2	0.677	208.5°	26.79°
1.3	0.701	159.2°	24.73°
1.4	0.721	130.45°	22.97°
1.67	0.763	89.66°	19.31°

6.5 Graphical Results

The program generates six figures, and I inspected each one visually after generating it.

Figure 7 shows the isentropic ratios and A/A^* on a logarithmic axis. Stagnation ratios grow monotonically with Mach number, while A/A^* has its minimum of 1 at $M = 1$ and rises on both sides, the double-valuedness that requires a branch choice in the inverse.

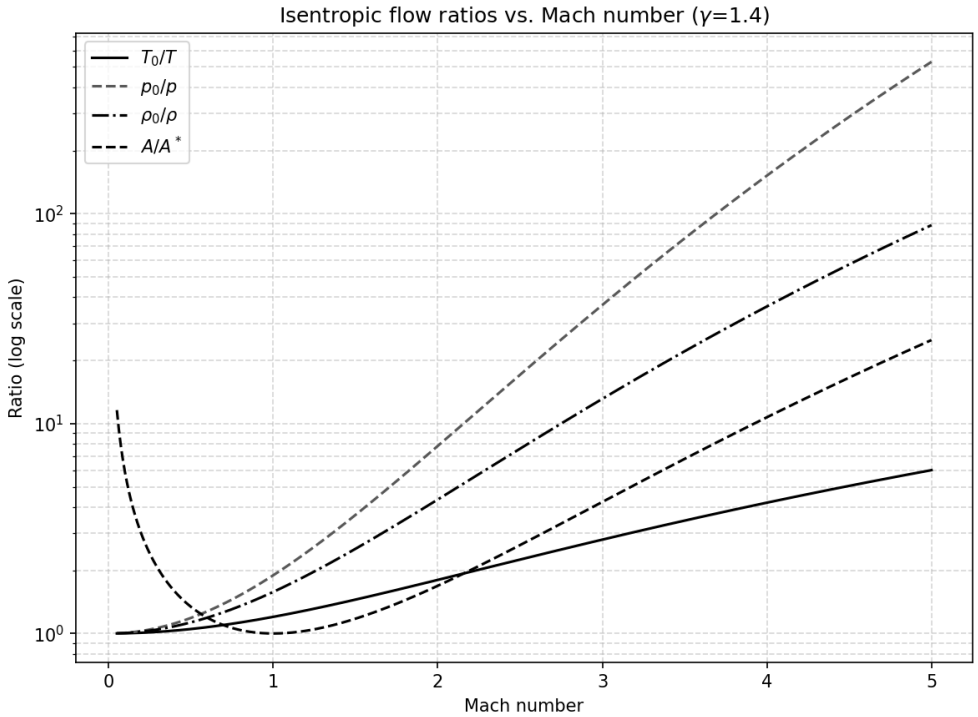


Figure 7: Isentropic stagnation-to-static ratios and area ratio A/A∗ versus Mach number (γ = 1.4).

Figure 8 shows the normal-shock relations. The post-shock Mach number falls toward 0.378, density saturates near 6, and the stagnation-pressure ratio collapses, quantifying the irreversibility of the shock.

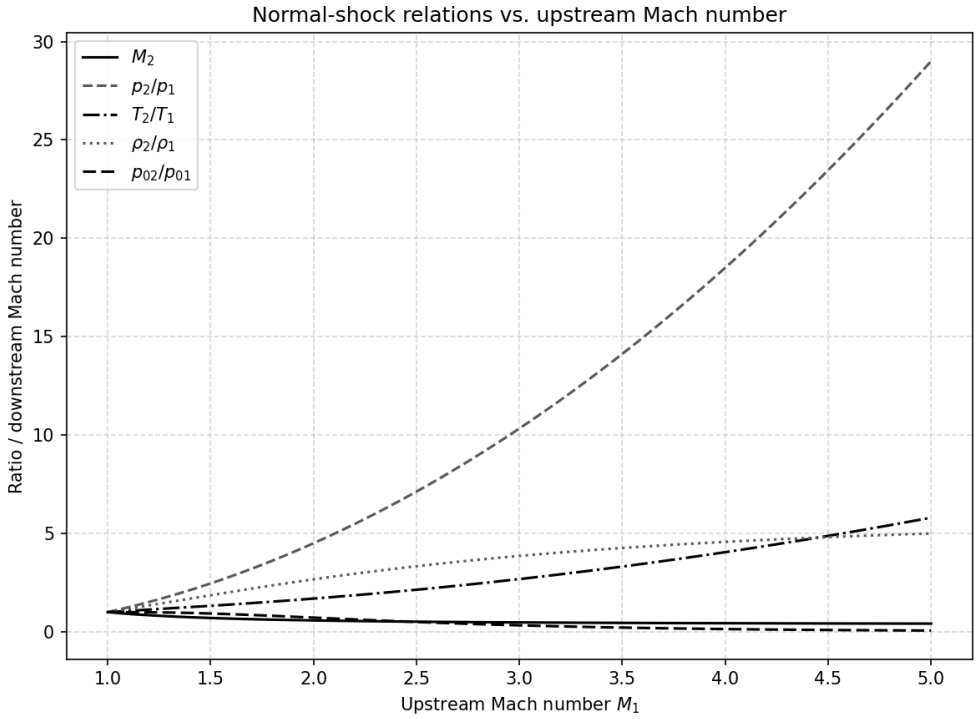


Figure 8: Normal-shock relations versus upstream Mach number M_1.

Figure 9 is the θ–β–M chart. Each curve starts at its Mach angle for θ = 0, rises to a nose at θ_{max} (marked), and returns to β = 90°. The lower branch of each curve is the weak solution and the upper branch the strong solution; the nose position moves to larger θ as M_1 increases.

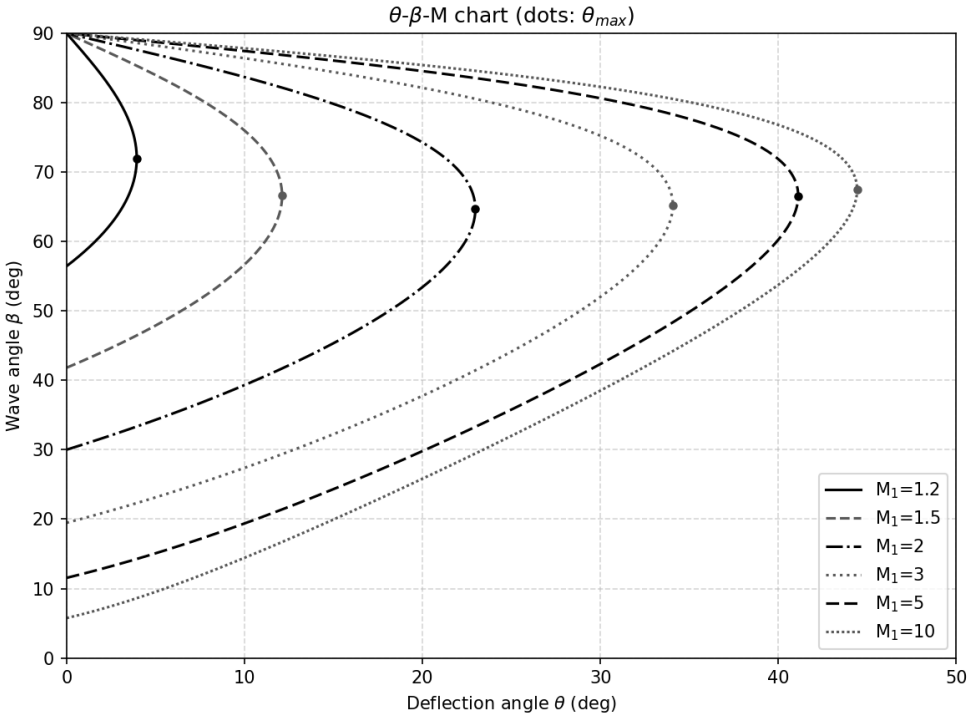


Figure 9: θ–β–M chart for γ = 1.4. Dots mark the maximum deflection angle θ_{max} for each Mach number.

Figure 10 shows the Prandtl–Meyer function approaching its limiting value ν_{max} = 130.45°, which sets the maximum turning angle before a vacuum forms.

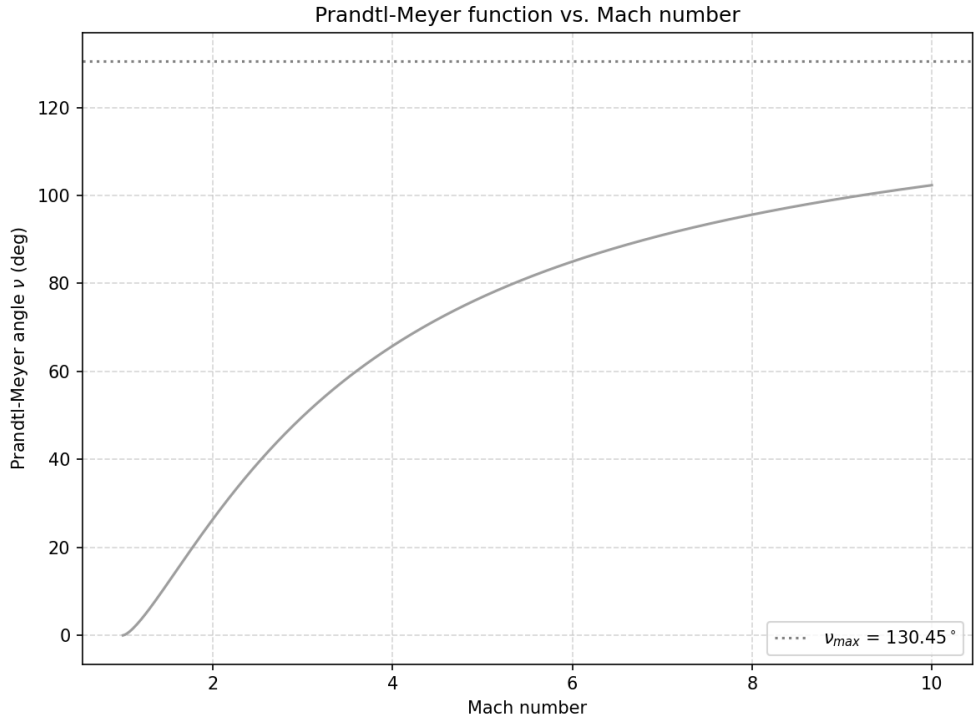


Figure 10: Prandtl–Meyer function ν(M) with its asymptotic limit ν_{max}.

Figure 11 and Figure 12 show the two duct-flow models. In both, all curves pass through the sonic reference value 1 at M = 1, which is the choked state.

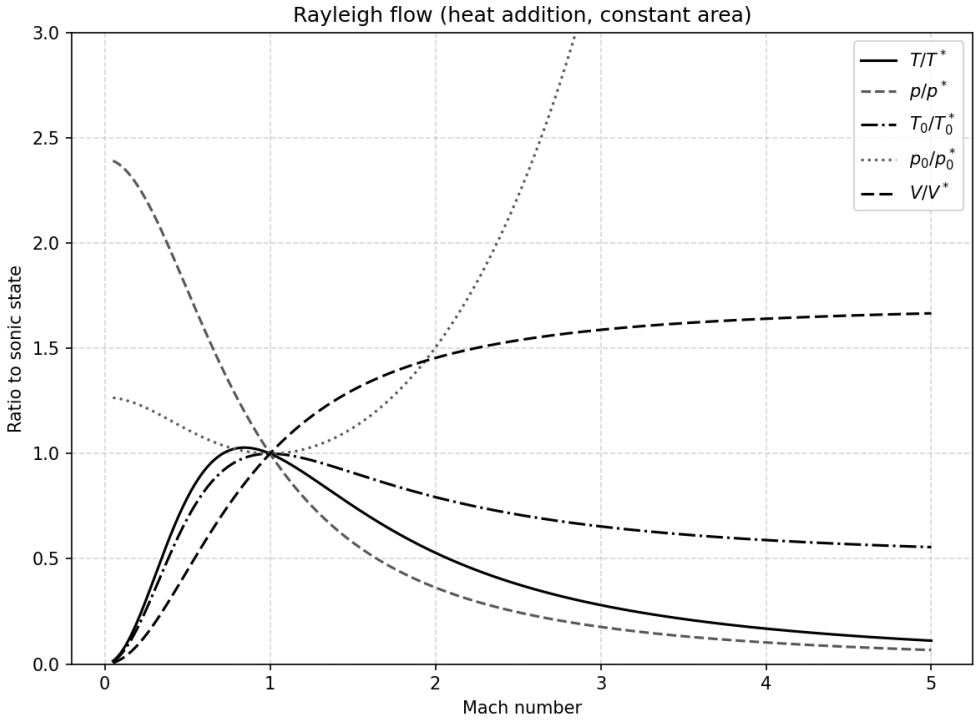


Figure 11: Rayleigh flow ratios relative to the sonic state versus Mach number.

In Figure 12, p_0/p_{0*} and $fL*/D$ reach their extremes of 1 and 0 at M = 1, and the supersonic $fL*/D$ flattens toward its limit of 0.8215.

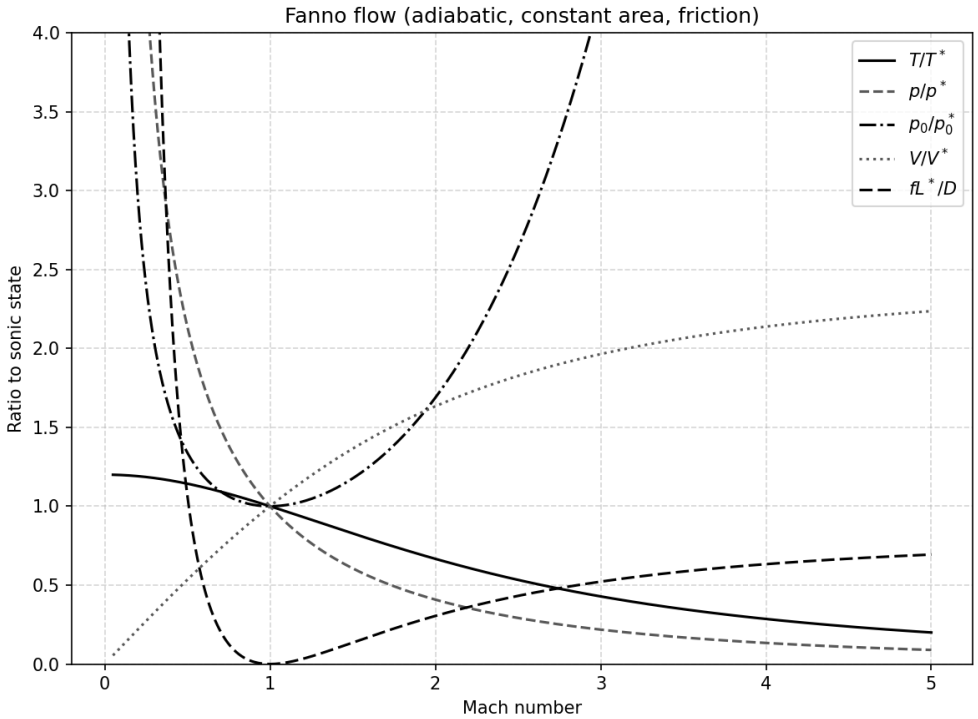


Figure 12: Fanno flow ratios and friction parameter fL∗/D relative to the sonic state versus Mach number.

7 Discussion

Correctness evidence. Confidence in the results rests on three independent lines of evidence: agreement with published tables at several Mach numbers for every relation family; satisfaction of physical identities (ideal-gas law, conservation of tangential velocity, $p_0/p_{0*} = A/A*$, reduction of the oblique shock to the normal shock at $\beta = 90°$); and round-trip consistency between forward and inverse functions. A bug that table spot-checks could not have found (Section 4.4) was caught by the third line, which argues for always keeping all three. The relations themselves can be cross-checked in the alternative derivations of Zucker and Biblarz [7] and John and Keith [8].

Limits of the verification. I entered the reference values in the tests from the published tables as I recalled them and cross-checked them by hand calculation; two entries turned out to be mis-typed or rounded (Section 4.3), and in both cases the code, not the reference, was correct. I have therefore not compared the tests against a scanned copy of the tables, and readers using the tool for safety-critical work should spot-check a few entries against NACA 1135 [2] themselves. Coverage of `main.py` (85 %) is lower than that of the library because interactive branches such as changing γ in the menu were not exercised.

Physical limits of the model. A calorically perfect gas is a good model for air up to roughly Mach 3 to 4 at ordinary temperatures. At higher Mach numbers vibrational excitation and dissociation make γ fall behind the shock, so values at $M_1 = 10$ in Table 9 should be read as the perfect-gas idealisation and not as predictions for real hypersonic flight. Likewise, the oblique-shock model is planar; flow over a cone requires the Taylor–Maccoll solution and gives different angles.

Design trade-offs. Named tuples were chosen over dataclasses to preserve backward compatibility with the original tuple-unpacking style. Raising exceptions for choked or detached conditions rather than returning NaN forces the caller to handle a physically important event instead of propagating it silently. The implementation relies on NumPy [10], SciPy [11] and Matplotlib [12], and the tests on pytest [13].

8 Conclusions and Future Work

8.1 Conclusions

1. The original calculator's formulas were correct, but I identified and fixed ten robustness and usability defects, including division by zero, silent acceptance of invalid inputs, and a failing inverse solver.
2. The tool now covers isentropic flow, normal shocks, oblique shocks (weak and strong), Prandtl–Meyer expansions, Rayleigh flow and Fanno flow, each with inverse relations and explicit choking or detachment detection.
3. A suite of 116 tests passes, giving 99 % coverage of the library, and all inverse solvers recover the Mach number to better than 3×10^{-13} relative error for $M \geq 1.001$.
4. Verification by round-trip sweep exposed a tolerance bug of up to 3.5×10^{-3} in code I had just written, which table comparisons could not detect; I fixed it and guarded against it with regression tests.
5. Worked examples reproduce the standard textbook behaviours: weak shocks are far more efficient than strong ones, heating and friction both drive duct flow toward M = 1, and expansion is limited by ν_{max}.

8.2 Future Work

- Conical (Taylor–Maccoll) shocks and shock reflection and interaction.
- Shock-in-duct solutions for Fanno flow beyond $L*$ and for converging–diverging nozzles with back pressure.
- Combined Rayleigh–Fanno flow and variable-area flow with friction and heat transfer.
- Real-gas and temperature-dependent γ options.
- Packaging for PyPI, continuous integration on GitHub Actions, and a graphical or web interface.

References

[1] J. D. Anderson, *Modern Compressible Flow: With Historical Perspective*, 3rd ed. New York: McGraw-Hill, 2003.

[2] Ames Research Staff, "Equations, Tables, and Charts for Compressible Flow," National Advisory Committee for Aeronautics, NACA Report 1135, 1953.

[3] H. W. Liepmann and A. Roshko, *Elements of Gasdynamics*. New York: Wiley, 1957 (Dover reprint, 2001).

[4] A. H. Shapiro, *The Dynamics and Thermodynamics of Compressible Fluid Flow*, vol. I. New York: Ronald Press, 1953.

[5] Th. Meyer, "Über zweidimensionale Bewegungsvorgänge in einem Gas, das mit Überschallgeschwindigkeit strömt," *Forschungsarbeiten auf dem Gebiete des Ingenieurwesens*, no. 62, VDI, 1908.

[6] Lord Rayleigh, "Aerial plane waves of finite amplitude," *Proceedings of the Royal Society of London A*, vol. 84, pp. 247–284, 1910.

[7] R. D. Zucker and O. Biblarz, *Fundamentals of Gas Dynamics*, 2nd ed. Hoboken: Wiley, 2002.

[8] J. E. A. John and T. G. Keith, *Gas Dynamics*, 3rd ed. Upper Saddle River: Pearson Prentice Hall, 2006.

[9] R. P. Brent, *Algorithms for Minimization Without Derivatives*. Englewood Cliffs: Prentice-Hall, 1973.

[10] C. R. Harris et al., "Array programming with NumPy," *Nature*, vol. 585, pp. 357–362, 2020.

[11] P. Virtanen et al., "SciPy 1.0: Fundamental algorithms for scientific computing in Python," *Nature Methods*, vol. 17, pp. 261–272, 2020.

[12] J. D. Hunter, "Matplotlib: A 2D graphics environment," *Computing in Science & Engineering*, vol. 9, no. 3, pp. 90–95, 2007.

[13] H. Krekel et al., *pytest documentation*. [Online]. Available: https://docs.pytest.org

Appendix A Running the Code

```
git clone https://github.com/sumniblue/Compressible_flow_calculator_Mach1.git
cd Compressible_flow_calculator_Mach1
pip install -r requirements.txt
python -m pytest -q                 # 116 tests
python main.py --validate           # 16 table checks
python main.py --mach 2.5           # one-shot summary
python main.py                      # validation, plots, interactive menu
```

Library use:

```
import compressible_flow as cf
cf.oblique_shock(3.0, theta_deg=15)             # weak solution
cf.oblique_shock(3.0, theta_deg=15, weak=False) # strong solution
cf.prandtl_meyer_expansion(2.0, 10)
cf.fanno_exit(0.5, 0.5)                         # raises ValueError if choked
```

Appendix B Summary of Changes to the Repository

Table 12: Changes made to the repository.

Item	Status
`main.py`	Rewritten as a command-line interface; calculation code moved out
`compressible_flow.py`	New: library with all relations
`test_compressible_flow.py`, `test_main.py`	New: 116 tests
`figures/`	New: six regenerated plots; the three PNGs in the repository root were replaced
`README.md`, `requirements.txt`, `.gitignore`	New

Appendix C Validation Output

Output of `python main.py --validate`:

```
======================================================================
VALIDATION AGAINST PUBLISHED TABLES (gamma = 1.4)
======================================================================
  [PASS] Isentropic M=2   T0/T              computed=1.8000  expected=1.8000
  [PASS] Isentropic M=2   p0/p              computed=7.8244  expected=7.8244
  [PASS] Isentropic M=2   A/A*              computed=1.6875  expected=1.6875
  [PASS] Normal shock M1=2  M2              computed=0.5774  expected=0.5774
  [PASS] Normal shock M1=2  p2/p1           computed=4.5000  expected=4.5000
  [PASS] Normal shock M1=2  T2/T1           computed=1.6875  expected=1.6875
  [PASS] Normal shock M1=2  p02/p01         computed=0.7209  expected=0.7209
  [PASS] Oblique M=2,theta=10  beta         computed=39.3139  expected=39.3100
  [PASS] Oblique M=2,theta=10  M2           computed=1.6405  expected=1.6410
  [PASS] Oblique M=2,theta=10  p2/p1        computed=1.7066  expected=1.7070
  [PASS] Prandtl-Meyer M=2  nu              computed=26.3798  expected=26.3800
  [PASS] Inverse PM nu=26.38 -> M           computed=2.0000  expected=2.0000
  [PASS] Rayleigh M=2  T0/T0*               computed=0.7934  expected=0.7934
  [PASS] Rayleigh M=2  p/p*                 computed=0.3636  expected=0.3636
  [PASS] Fanno M=2  fL*/D                   computed=0.3050  expected=0.3050
  [PASS] Fanno M=2  p/p*                    computed=0.4082  expected=0.4082
----------------------------------------------------------------------
  16/16 checks passed
======================================================================
```

Appendix D compressible_flow.py (library)

This appendix lists the complete final source of the numerical library, with line numbers. All 116 tests (Appendices F and G) run against exactly this version.

```
  1  """
  2  compressible_flow.py
  3  ====================
  4
  5  One-dimensional compressible-flow relations for a calorically perfect gas.
  6
  7  Contents
  8  --------
  9  * Isentropic flow            : static/stagnation ratios, A/A*, inverse A/A* -> M
 10  * Normal shock               : downstream state, inverse relations, Pitot ratio
 11  * Oblique shock              : theta-beta-M relation, weak/strong solutions,
 12                                 maximum deflection, downstream state
 13  * Prandtl-Meyer expansion    : nu(M), inverse, expansion-fan solver
 14  * Rayleigh flow              : frictionless duct with heat addition
 15  * Fanno flow                 : adiabatic duct with wall friction
 16
 17  All angles at the public interface are in DEGREES. All functions validate their
 18  inputs and raise ``ValueError`` for physically meaningless requests (e.g. a
 19  normal shock in subsonic flow, or a wedge angle larger than the detachment
 20  limit). Forward relations accept scalars or NumPy arrays; inverse (root-finding)
 21  functions accept scalars only.
 22
 23  References
 24  ----------
 25  [1] Anderson, J.D., Modern Compressible Flow, 3rd ed., McGraw-Hill, 2003.
 26  [2] Ames Research Staff, NACA Report 1135, "Equations, Tables, and Charts for
 27      Compressible Flow", 1953.
 28  [3] Liepmann, H.W. and Roshko, A., Elements of Gasdynamics, Wiley, 1957.
 29  """
 30
 31  from __future__ import annotations
 32
 33  from typing import NamedTuple
 34
 35  import numpy as np
 36  from scipy.optimize import brentq
 37
 38  DEFAULT_GAMMA = 1.4
 39
 40
 41  # ---------------------------------------------------------------------------- #
 42  # Result containers (NamedTuples can still be unpacked like plain tuples)
 43  # ---------------------------------------------------------------------------- #
 44  class IsentropicState(NamedTuple):
 45      T0_T: float
 46      p0_p: float
 47      rho0_rho: float
 48
 49
 50  class NormalShockState(NamedTuple):
 51      M2: float
 52      p2_p1: float
 53      rho2_rho1: float
 54      T2_T1: float
 55      p02_p01: float
 56
 57
 58  class ObliqueShockState(NamedTuple):
 59      beta_deg: float
 60      theta_deg: float
 61      Mn1: float
 62      Mn2: float
 63      M2: float
 64      p2_p1: float
 65      rho2_rho1: float
 66      T2_T1: float
 67      p02_p01: float
 68
```

```
 69
 70  class ExpansionState(NamedTuple):
 71      M2: float
 72      nu1_deg: float
 73      nu2_deg: float
 74      p2_p1: float
 75      rho2_rho1: float
 76      T2_T1: float
 77      mu1_deg: float  # leading Mach wave angle
 78      mu2_deg: float  # trailing Mach wave angle
 79
 80
 81  class RayleighState(NamedTuple):
 82      T_Tstar: float
 83      p_pstar: float
 84      rho_rhostar: float
 85      V_Vstar: float
 86      T0_T0star: float
 87      p0_p0star: float
 88
 89
 90  class FannoState(NamedTuple):
 91      T_Tstar: float
 92      p_pstar: float
 93      rho_rhostar: float
 94      V_Vstar: float
 95      p0_p0star: float
 96      fLstar_D: float  # Darcy friction factor * L* / D  (= 4 * Fanning f * L*/D)
 97
 98
 99  # ---------------------------------------------------------------------------- #
100  # Validation helpers
101  # ---------------------------------------------------------------------------- #
102  def _check_gamma(gamma):
103      g = np.asarray(gamma, dtype=float)
104      if g.ndim != 0 or not np.isfinite(g) or g <= 1.0:
105          raise ValueError(f"gamma must be a finite scalar > 1, got {gamma!r}.")
106      return float(g)
107
108
109  def _check_mach(M, *, minimum=0.0, inclusive=False, name="Mach number"):
110      """Return M as a float array; require M > minimum (or >= if inclusive)."""
111      arr = np.asarray(M, dtype=float)
112      if not np.all(np.isfinite(arr)):
113          raise ValueError(f"{name} must be finite, got {M!r}.")
114      bad = np.any(arr < minimum) if inclusive else np.any(arr <= minimum)
115      if bad:
116          rel = ">=" if inclusive else ">"
117          raise ValueError(f"{name} must be {rel} {minimum}, got {M!r}.")
118      return arr
119
120
121  def _scalar(x):
122      """Collapse 0-d arrays to Python floats, leave real arrays alone."""
123      x = np.asarray(x)
124      return float(x) if x.ndim == 0 else x
125
126
127  # ---------------------------------------------------------------------------- #
128  # 1. Isentropic flow
129  # ---------------------------------------------------------------------------- #
130  def isentropic_ratios(M, gamma=DEFAULT_GAMMA) -> IsentropicState:
131      """Stagnation-to-static ratios T0/T, p0/p, rho0/rho at Mach M (M >= 0)."""
132      g = _check_gamma(gamma)
133      M = _check_mach(M, inclusive=True)
134      factor = 1.0 + 0.5 * (g - 1.0) * M**2
135      return IsentropicState(
136          _scalar(factor),
137          _scalar(factor ** (g / (g - 1.0))),
138          _scalar(factor ** (1.0 / (g - 1.0))),
139      )
140
141
142  def area_mach_ratio(M, gamma=DEFAULT_GAMMA):
143      """Area ratio A/A* for isentropic quasi-1D flow (M > 0)."""
144      g = _check_gamma(gamma)
145      M = _check_mach(M)
```

```
146      term = (2.0 / (g + 1.0)) * (1.0 + 0.5 * (g - 1.0) * M**2)
147      return _scalar(term ** ((g + 1.0) / (2.0 * (g - 1.0))) / M)
148
149
150  def mach_from_area_ratio(A_Astar, supersonic=True, gamma=DEFAULT_GAMMA):
151      """Invert A/A* -> M. ``supersonic`` selects the branch (M>1 or M<1)."""
152      g = _check_gamma(gamma)
153      A = float(A_Astar)
154      if not np.isfinite(A) or A < 1.0 - 1e-12:
155          raise ValueError(f"A/A* must be >= 1, got {A_Astar!r}.")
156      # NB: A/A* - 1 ~ (M-1)^2 near the sonic point, so the tolerance must be a pure
157      # absolute one (np.isclose's default rtol would corrupt M by ~1e-3 here).
158      if abs(A - 1.0) <= 1e-12:
159          return 1.0
160      f = lambda m: area_mach_ratio(m, g) - A
161      if supersonic:
162          hi = 2.0
163          while f(hi) < 0.0:
164              hi *= 2.0
165          return brentq(f, 1.0, hi, xtol=1e-14)
166      return brentq(f, 1e-9, 1.0, xtol=1e-14)
167
168
169  # ------------------------------------------------------------------------ #
170  # 2. Normal shock
171  # ------------------------------------------------------------------------ #
172  def normal_shock(M1, gamma=DEFAULT_GAMMA) -> NormalShockState:
173      """Property ratios across a stationary normal shock (M1 >= 1)."""
174      g = _check_gamma(gamma)
175      M1 = _check_mach(M1, minimum=1.0, inclusive=True, name="Upstream Mach number M1")
176      M1sq = M1**2
177      M2 = np.sqrt((1 + 0.5 * (g - 1) * M1sq) / (g * M1sq - 0.5 * (g - 1)))
178      p2_p1 = 1 + (2 * g / (g + 1)) * (M1sq - 1)
179      rho2_rho1 = ((g + 1) * M1sq) / ((g - 1) * M1sq + 2)
180      T2_T1 = p2_p1 / rho2_rho1
181      p02_p01 = (rho2_rho1 ** (g / (g - 1))) * (
182          ((g + 1) / (2 * g * M1sq - (g - 1))) ** (1 / (g - 1))
183      )
184      return NormalShockState(*(_scalar(v) for v in (M2, p2_p1, rho2_rho1, T2_T1,
     p02_p01)))
185
186
187  def shock_entropy_rise(M1, gamma=DEFAULT_GAMMA):
188      """Dimensionless entropy rise (s2 - s1)/R = -ln(p02/p01) across a normal shock."""
189      return _scalar(-np.log(normal_shock(M1, gamma).p02_p01))
190
191
192  def pitot_pressure_ratio(M1, gamma=DEFAULT_GAMMA):
193      """Rayleigh Pitot formula: p02/p1 for a Pitot probe in supersonic flow (M1 >=
     1)."""
194      g = _check_gamma(gamma)
195      M1 = _check_mach(M1, minimum=1.0, inclusive=True, name="Upstream Mach number M1")
196      M1sq = M1**2
197      a = (0.5 * (g + 1) * M1sq) ** (g / (g - 1))
198      b = (2 * g * M1sq / (g + 1) - (g - 1) / (g + 1)) ** (1 / (g - 1))
199      return _scalar(a / b)
200
201
202  def mach1_from_pressure_ratio(p2_p1, gamma=DEFAULT_GAMMA):
203      """Upstream Mach number that produces a given static-pressure jump p2/p1 >= 1."""
204      g = _check_gamma(gamma)
205      r = float(p2_p1)
206      if not np.isfinite(r) or r < 1.0:
207          raise ValueError(f"p2/p1 across a shock must be >= 1, got {p2_p1!r}.")
208      return float(np.sqrt(1.0 + (g + 1.0) / (2.0 * g) * (r - 1.0)))
209
210
211  def mach1_from_downstream_mach(M2, gamma=DEFAULT_GAMMA):
212      """Upstream Mach number for a given post-shock Mach number M2."""
213      g = _check_gamma(gamma)
214      M2 = float(M2)
215      lo = np.sqrt((g - 1.0) / (2.0 * g))
216      if not (lo <= M2 <= 1.0):
217          raise ValueError(
218              f"Post-shock Mach number must lie in [{lo:.4f}, 1] for gamma={g}, got
     {M2!r}."
219          )
```

```
220     return float(np.sqrt((1 + 0.5 * (g - 1) * M2**2) / (g * M2**2 - 0.5 * (g - 1))))
221
222
223 # ------------------------------------------------------------------------ #
224 # 3. Oblique shock
225 # ------------------------------------------------------------------------ #
226 def mach_angle_deg(M):
227     """Mach angle mu = arcsin(1/M) in degrees (M >= 1)."""
228     M = _check_mach(M, minimum=1.0, inclusive=True)
229     return _scalar(np.degrees(np.arcsin(1.0 / M)))
230
231
232 def theta_from_beta(M1, beta_deg, gamma=DEFAULT_GAMMA):
233     """Flow-deflection angle theta (deg) for shock-wave angle beta (deg): the theta-
    beta-M relation."""
234     g = _check_gamma(gamma)
235     M1 = _check_mach(M1, minimum=1.0, inclusive=True, name="Upstream Mach number M1")
236     beta = np.radians(np.asarray(beta_deg, dtype=float))
237     tan_theta = (
238         2.0
239         / np.tan(beta)
240         * (M1**2 * np.sin(beta) ** 2 - 1.0)
241         / (M1**2 * (g + np.cos(2.0 * beta)) + 2.0)
242     )
243     return _scalar(np.degrees(np.arctan(tan_theta)))
244
245
246 def max_deflection_angle(M1, gamma=DEFAULT_GAMMA):
247     """Return (theta_max_deg, beta_at_theta_max_deg) for an attached oblique shock."""
248     g = _check_gamma(gamma)
249     M1 = float(_check_mach(M1, minimum=1.0, name="Upstream Mach number M1 (must be >
    1)"))
250     M2sq = M1**2
251     sin2b = (1.0 / (g * M2sq)) * (
252         0.25 * (g + 1) * M2sq
253         - 1.0
254         + np.sqrt((g + 1) * (1 + 0.5 * (g - 1) * M2sq + (g + 1) / 16.0 * M2sq**2))
255     )
256     beta_max = np.arcsin(np.sqrt(sin2b))
257     return float(theta_from_beta(M1, np.degrees(beta_max), g)),
    float(np.degrees(beta_max))
258
259
260 def oblique_shock(M1, theta_deg=None, beta_deg=None, weak=True, gamma=DEFAULT_GAMMA) ->
    ObliqueShockState:
261     """
262     Solve an oblique shock given M1 and EITHER the deflection angle ``theta_deg``
263     or the wave angle ``beta_deg`` (exactly one must be supplied).
264
265     ``weak`` chooses the weak (default) or strong solution when theta is given.
266     Raises ValueError if theta exceeds the detachment limit theta_max(M1).
267     """
268     g = _check_gamma(gamma)
269     M1 = float(_check_mach(M1, minimum=1.0, name="Upstream Mach number M1 (must be >
    1)"))
270     if (theta_deg is None) == (beta_deg is None):
271         raise ValueError("Provide exactly one of theta_deg or beta_deg.")
272
273     mu = float(mach_angle_deg(M1))
274     if beta_deg is None:
275         theta = float(theta_deg)
276         if theta < 0.0:
277             raise ValueError("theta must be >= 0 (mirror the geometry for
    expansions).")
278         theta_max, beta_tmax = max_deflection_angle(M1, g)
279         if theta > theta_max + 1e-12:
280             raise ValueError(
281                 f"theta = {theta:.3f} deg exceeds theta_max = {theta_max:.3f} deg at "
282                 f"M1 = {M1:g}: the shock detaches (no attached oblique solution)."
283             )
284         if theta == 0.0:
285             beta = mu if weak else 90.0
286         else:
287             f = lambda b: float(theta_from_beta(M1, b, g)) - theta
288             if weak:
289                 beta = brentq(f, mu, beta_tmax, xtol=1e-13)
290             else:
```

```
291                 beta = brentq(f, beta_tmax, 90.0, xtol=1e-13)
292     else:
293         beta = float(beta_deg)
294         if not (mu - 1e-12 <= beta <= 90.0 + 1e-12):
295             raise ValueError(
296                 f"beta must lie between the Mach angle ({mu:.3f} deg) and 90 deg, got
    {beta:g}."
297             )
298         theta = float(theta_from_beta(M1, beta, g))
299 
300     b = np.radians(beta)
301     Mn1 = max(M1 * np.sin(b), 1.0)
302     ns = normal_shock(Mn1, g)
303     M2 = ns.M2 / np.sin(b - np.radians(theta))
304     return ObliqueShockState(
305         float(beta), float(theta), float(Mn1), float(ns.M2), float(M2),
306         float(ns.p2_p1), float(ns.rho2_rho1), float(ns.T2_T1), float(ns.p02_p01),
307     )
308 
309 
310 # ---------------------------------------------------------------------------- #
311 # 4. Prandtl-Meyer expansion
312 # ---------------------------------------------------------------------------- #
313 def prandtl_meyer(M, gamma=DEFAULT_GAMMA):
314     """Prandtl-Meyer function nu(M) in degrees (M >= 1)."""
315     g = _check_gamma(gamma)
316     M = _check_mach(M, minimum=1.0, inclusive=True)
317     a = np.sqrt((g + 1) / (g - 1))
318     s = np.sqrt(M**2 - 1)
319     nu = a * np.arctan(np.sqrt((g - 1) / (g + 1)) * s) - np.arctan(s)
320     return _scalar(np.degrees(nu))
321 
322 
323 def prandtl_meyer_max(gamma=DEFAULT_GAMMA):
324     """Limiting turn angle nu_max = (pi/2)(sqrt((g+1)/(g-1)) - 1), in degrees."""
325     g = _check_gamma(gamma)
326     return float(90.0 * (np.sqrt((g + 1) / (g - 1)) - 1.0))
327 
328 
329 def mach_from_pm_angle(nu_target_deg, gamma=DEFAULT_GAMMA):
330     """Invert the Prandtl-Meyer function: Mach number whose nu(M) equals the target."""
331     g = _check_gamma(gamma)
332     nu = float(nu_target_deg)
333     nu_max = prandtl_meyer_max(g)
334     if not np.isfinite(nu) or nu < 0.0:
335         raise ValueError(f"nu must be >= 0, got {nu_target_deg!r}.")
336     if nu >= nu_max:
337         raise ValueError(
338             f"nu = {nu:g} deg is not reachable: the limit for gamma={g:g} is
    {nu_max:.4f} deg."
339         )
340     if nu == 0.0:
341         return 1.0
342     f = lambda m: float(prandtl_meyer(m, g)) - nu
343     hi = 2.0
344     while f(hi) < 0.0:
345         hi *= 2.0
346         if hi > 1e12:
347             raise ValueError("Could not bracket the Mach number (nu too close to the
    limit).")
348     return brentq(f, 1.0, hi, xtol=1e-13)
349 
350 
351 def prandtl_meyer_expansion(M1, turn_deg, gamma=DEFAULT_GAMMA) -> ExpansionState:
352     """Isentropic expansion of supersonic flow M1 turned through ``turn_deg``
    degrees."""
353     g = _check_gamma(gamma)
354     M1 = float(_check_mach(M1, minimum=1.0, inclusive=True, name="Upstream Mach number
    M1"))
355     turn = float(turn_deg)
356     if turn < 0.0:
357         raise ValueError("Expansion turn angle must be >= 0.")
358     nu1 = float(prandtl_meyer(M1, g))
359     nu2 = nu1 + turn
360     M2 = mach_from_pm_angle(nu2, g)
361     t1, p1, r1 = isentropic_ratios(M1, g)
362     t2, p2, r2 = isentropic_ratios(M2, g)
```

```
363     return ExpansionState(
364         M2, nu1, nu2,
365         p1 / p2, r1 / r2, t1 / t2,  # static ratios: (p0/p)_1 / (p0/p)_2 etc.
366         float(mach_angle_deg(M1)), float(mach_angle_deg(M2)),
367     )
368 
369 
370 # ------------------------------------------------------------------------ #
371 # 5. Rayleigh flow (frictionless, constant-area, heat addition)
372 # ------------------------------------------------------------------------ #
373 def rayleigh_ratios(M, gamma=DEFAULT_GAMMA) -> RayleighState:
374     """Rayleigh-flow ratios relative to the sonic (*) state, M > 0."""
375     g = _check_gamma(gamma)
376     M = _check_mach(M)
377     d = 1.0 + g * M**2
378     T_Ts = (g + 1) ** 2 * M**2 / d**2
379     p_ps = (g + 1) / d
380     rho_rs = d / ((g + 1) * M**2)
381     T0_T0s = (g + 1) * M**2 * (2 + (g - 1) * M**2) / d**2
382     p0_p0s = p_ps * ((2 + (g - 1) * M**2) / (g + 1)) ** (g / (g - 1))
383     return RayleighState(*(_scalar(v) for v in (T_Ts, p_ps, rho_rs, 1.0 / rho_rs,
    T0_T0s, p0_p0s)))
384 
385 
386 def mach_from_rayleigh_T0_ratio(T0_T0star, supersonic, gamma=DEFAULT_GAMMA):
387     """Invert T0/T0* -> M on the subsonic or supersonic branch."""
388     g = _check_gamma(gamma)
389     r = float(T0_T0star)
390     if not (0.0 < r <= 1.0 + 1e-12):
391         raise ValueError(f"T0/T0* must be in (0, 1], got {T0_T0star!r}.")
392     if abs(r - 1.0) <= 1e-12:  # pure absolute tolerance: T0/T0* - 1 ~ (M-1)^2 near M =
    1
393         return 1.0
394     f = lambda m: float(rayleigh_ratios(m, g).T0_T0star) - r
395     if supersonic:
396         floor = (g + 1) * (g - 1) / g**2  # limit as M -> infinity
397         if r <= floor:
398             raise ValueError(f"No supersonic solution: T0/T0* must exceed
    {floor:.4f}.")
399         hi = 2.0
400         while f(hi) > 0.0:
401             hi *= 2.0
402         return brentq(f, 1.0, hi, xtol=1e-14)
403     return brentq(f, 1e-9, 1.0, xtol=1e-14)
404 
405 
406 def rayleigh_exit(M1, T02_T01, gamma=DEFAULT_GAMMA):
407     """
408     Exit Mach number after heating (T02/T01 > 1) or cooling (< 1) a Rayleigh duct.
409     The flow stays on its original branch; heating beyond the sonic limit
410     raises ValueError (thermal choking).
411     """
412     g = _check_gamma(gamma)
413     M1 = float(_check_mach(M1))
414     ratio = float(T02_T01)
415     if ratio <= 0.0:
416         raise ValueError("T02/T01 must be positive.")
417     target = float(rayleigh_ratios(M1, g).T0_T0star) * ratio
418     if target > 1.0 + 1e-12:
419         max_ratio = 1.0 / float(rayleigh_ratios(M1, g).T0_T0star)
420         raise ValueError(
421             f"Thermally choked: maximum T02/T01 for M1={M1:g} is {max_ratio:.4f}, got
    {ratio:g}."
422         )
423     return mach_from_rayleigh_T0_ratio(min(target, 1.0), supersonic=M1 > 1.0, gamma=g)
424 
425 
426 # ------------------------------------------------------------------------ #
427 # 6. Fanno flow (adiabatic, constant-area, wall friction)
428 # ------------------------------------------------------------------------ #
429 def fanno_parameter(M, gamma=DEFAULT_GAMMA):
430     """Friction parameter f*L*/D (Darcy f, i.e. 4 x Fanning) needed to reach M = 1."""
431     g = _check_gamma(gamma)
432     M = _check_mach(M)
433     M2 = M**2
434     val = (1 - M2) / (g * M2) + (g + 1) / (2 * g) * np.log((g + 1) * M2 / (2 + (g - 1)
    * M2))
```

```
435      return _scalar(val)
436
437
438  def fanno_ratios(M, gamma=DEFAULT_GAMMA) -> FannoState:
439      """Fanno-flow ratios relative to the sonic (*) state, M > 0."""
440      g = _check_gamma(gamma)
441      M = _check_mach(M)
442      e = 2 + (g - 1) * M**2
443      T_Ts = (g + 1) / e
444      p_ps = np.sqrt((g + 1) / e) / M
445      rho_rs = np.sqrt(e / (g + 1)) / M
446      p0_p0s = (e / (g + 1)) ** ((g + 1) / (2 * (g - 1))) / M
447      return FannoState(
448          *(_scalar(v) for v in (T_Ts, p_ps, rho_rs, 1.0 / rho_rs, p0_p0s)),
449          fanno_parameter(M, g),
450      )
451
452
453  def mach_from_fanno_parameter(fLstar_D, supersonic, gamma=DEFAULT_GAMMA):
454      """Invert f*L*/D -> M on the subsonic or supersonic branch."""
455      g = _check_gamma(gamma)
456      p = float(fLstar_D)
457      if p < 0.0 or not np.isfinite(p):
458          raise ValueError(f"f L*/D must be >= 0, got {fLstar_D!r}.")
459      if p == 0.0:
460          return 1.0
461      f = lambda m: float(fanno_parameter(m, g)) - p
462      if supersonic:
463          pmax = (g + 1) / (2 * g) * np.log((g + 1) / (g - 1)) - 1.0 / g
464          if p >= pmax:
465              raise ValueError(f"No supersonic solution: f L*/D must be below
     {pmax:.4f}.")
466          hi = 2.0
467          while f(hi) < 0.0:
468              hi *= 2.0
469          return brentq(f, 1.0, hi, xtol=1e-14)
470      return brentq(f, 1e-9, 1.0, xtol=1e-14)
471
472
473  def fanno_exit(M1, fL_D, gamma=DEFAULT_GAMMA):
474      """
475      Exit Mach number of a Fanno duct with friction parameter f*L/D (Darcy f).
476      Raises ValueError if the duct is longer than L*, i.e. the flow chokes.
477      """
478      g = _check_gamma(gamma)
479      M1 = float(_check_mach(M1))
480      if fL_D < 0.0:
481          raise ValueError("f L/D must be >= 0.")
482      remaining = float(fanno_parameter(M1, g)) - float(fL_D)
483      if remaining < -1e-12:
484          raise ValueError(
485              f"Choked: duct length f L/D = {fL_D:g} exceeds f L*/D = "
486              f"{float(fanno_parameter(M1, g)):.4f} for M1 = {M1:g}."
487          )
488      return mach_from_fanno_parameter(max(remaining, 0.0), supersonic=M1 > 1.0, gamma=g)
```

Appendix E main.py (command-line interface)

Complete final source of the command-line front end: self-validation, plots, one-shot summary and interactive menu.

```
"""
main.py - command-line front end for the Compressible Flow Calculator.

Usage
-----
    python main.py                      # validate, make plots, then interactive menu
    python main.py --validate           # run the self-validation only
    python main.py --plots              # (re)generate the figures only
    python main.py --mach 2.0 [--gamma 1.4]   # one-shot summary, no prompts
    python main.py --no-plots --no-interactive
    python main.py --show               # also open plot windows (default: save only)
"""

from __future__ import annotations

import argparse
import os
import sys

import numpy as np

import compressible_flow as cf

FIG_DIR = "figures"


# ---------------------------------------------------------------------------- #
# Self-validation (asserts, so a regression fails loudly)
# ---------------------------------------------------------------------------- #
def run_validation(verbose=True) -> bool:
    checks = []

    def check(label, computed, expected, tol):
        ok = abs(computed - expected) <= tol
        checks.append(ok)
        if verbose:
            print(f"  [{'PASS' if ok else 'FAIL'}] {label:<34} computed={computed:.4f}
expected={expected:.4f}")

    if verbose:
        print("=" * 72)
        print("VALIDATION AGAINST PUBLISHED TABLES (gamma = 1.4)")
        print("=" * 72)

    s = cf.isentropic_ratios(2.0)
    check("Isentropic M=2   T0/T", s.T0_T, 1.8000, 5e-5)
    check("Isentropic M=2   p0/p", s.p0_p, 7.8244, 5e-4)
    check("Isentropic M=2   A/A*", cf.area_mach_ratio(2.0), 1.6875, 5e-5)

    n = cf.normal_shock(2.0)
    check("Normal shock M1=2  M2", n.M2, 0.5774, 5e-5)
    check("Normal shock M1=2  p2/p1", n.p2_p1, 4.5000, 5e-5)
    check("Normal shock M1=2  T2/T1", n.T2_T1, 1.6875, 5e-5)
    check("Normal shock M1=2  p02/p01", n.p02_p01, 0.7209, 5e-5)

    o = cf.oblique_shock(2.0, theta_deg=10.0)
    check("Oblique M=2,theta=10  beta", o.beta_deg, 39.31, 0.01)
    check("Oblique M=2,theta=10  M2", o.M2, 1.641, 2e-3)
    check("Oblique M=2,theta=10  p2/p1", o.p2_p1, 1.707, 2e-3)

    check("Prandtl-Meyer M=2  nu", cf.prandtl_meyer(2.0), 26.380, 2e-3)
    check("Inverse PM nu=26.38 -> M", cf.mach_from_pm_angle(26.380), 2.0, 1e-3)

    r = cf.rayleigh_ratios(2.0)
    check("Rayleigh M=2  T0/T0*", r.T0_T0star, 0.7934, 5e-5)
    check("Rayleigh M=2  p/p*", r.p_pstar, 0.3636, 5e-5)

    f = cf.fanno_ratios(2.0)
```

```
 68      check("Fanno M=2  fL*/D", f.fLstar_D, 0.3050, 5e-5)
 69      check("Fanno M=2  p/p*", f.p_pstar, 0.4082, 5e-5)
 70
 71      ok = all(checks)
 72      if verbose:
 73          print("-" * 72)
 74          print(f"  {sum(checks)}/{len(checks)} checks passed")
 75          print("=" * 72 + "\n")
 76      return ok
 77
 78
 79  # ---------------------------------------------------------------------------- #
 80  # Plots
 81  # ---------------------------------------------------------------------------- #
 82  def _finish(fig, plt, name, show):
 83      os.makedirs(FIG_DIR, exist_ok=True)
 84      fig.tight_layout()
 85      fig.savefig(os.path.join(FIG_DIR, name), dpi=150)
 86      if show:
 87          plt.show()
 88      plt.close(fig)
 89
 90
 91  def make_plots(gamma=1.4, show=False):
 92      import matplotlib
 93      if not show:
 94          matplotlib.use("Agg")
 95      import matplotlib.pyplot as plt
 96
 97      grid = dict(linestyle="--", alpha=0.5)
 98
 99      # 1. isentropic ratios + A/A*
100      M = np.linspace(0.05, 5, 400)
101      s = cf.isentropic_ratios(M, gamma)
102      fig, ax = plt.subplots(figsize=(8, 6))
103      ax.plot(M, s.T0_T, label=r"$T_0/T$")
104      ax.plot(M, s.p0_p, label=r"$p_0/p$")
105      ax.plot(M, s.rho0_rho, label=r"$\rho_0/\rho$")
106      ax.plot(M, cf.area_mach_ratio(M, gamma), "k--", label=r"$A/A^*$")
107      ax.set_yscale("log")
108      ax.set(xlabel="Mach number", ylabel="Ratio (log scale)",
109             title=f"Isentropic flow ratios vs. Mach number ($\\gamma$={gamma})")
110      ax.legend(); ax.grid(True, which="both", **grid)
111      _finish(fig, plt, "isentropic_curves.png", show)
112
113      # 2. normal shock
114      M1 = np.linspace(1, 5, 400)
115      n = cf.normal_shock(M1, gamma)
116      fig, ax = plt.subplots(figsize=(8, 6))
117      ax.plot(M1, n.M2, label=r"$M_2$")
118      ax.plot(M1, n.p2_p1, label=r"$p_2/p_1$")
119      ax.plot(M1, n.T2_T1, label=r"$T_2/T_1$")
120      ax.plot(M1, n.rho2_rho1, label=r"$\rho_2/\rho_1$")
121      ax.plot(M1, n.p02_p01, label=r"$p_{02}/p_{01}$")
122      ax.set(xlabel="Upstream Mach number $M_1$", ylabel="Ratio / downstream Mach
    number",
123             title="Normal-shock relations vs. upstream Mach number")
124      ax.legend(); ax.grid(True, **grid)
125      _finish(fig, plt, "normal_shock_curves.png", show)
126
127      # 3. Prandtl-Meyer
128      M = np.linspace(1, 10, 400)
129      fig, ax = plt.subplots(figsize=(8, 6))
130      ax.plot(M, cf.prandtl_meyer(M, gamma), color="darkorange")
131      ax.axhline(cf.prandtl_meyer_max(gamma), color="gray", ls=":",
132                 label=rf"$\nu_{{max}}$ = {cf.prandtl_meyer_max(gamma):.2f}$^\circ$")
133      ax.set(xlabel="Mach number", ylabel=r"Prandtl-Meyer angle $\nu$ (deg)",
134             title="Prandtl-Meyer function vs. Mach number")
135      ax.legend(); ax.grid(True, **grid)
136      _finish(fig, plt, "prandtl_meyer_curve.png", show)
137
138      # 4. theta-beta-M chart
139      fig, ax = plt.subplots(figsize=(8, 6))
140      for Mach in (1.2, 1.5, 2, 3, 5, 10):
141          mu = float(cf.mach_angle_deg(Mach))
142          beta = np.linspace(mu, 90, 500)
143          theta = cf.theta_from_beta(Mach, beta, gamma)
```

```
144         line, = ax.plot(theta, beta, label=f"M$_1$={Mach}")
145         tmax, bmax = cf.max_deflection_angle(Mach, gamma)
146         ax.plot(tmax, bmax, "o", color=line.get_color(), ms=4)
147     ax.set(xlabel=r"Deflection angle $\theta$ (deg)", ylabel=r"Wave angle
    $\beta$ (deg)",
148            title=r"$\theta$-$\beta$-M chart (dots: $\theta_{max}$)", xlim=(0, 50),
    ylim=(0, 90))
149     ax.legend(); ax.grid(True, **grid)
150     _finish(fig, plt, "theta_beta_M_chart.png", show)
151
152     # 5. Rayleigh
153     M = np.linspace(0.05, 5, 400)
154     r = cf.rayleigh_ratios(M, gamma)
155     fig, ax = plt.subplots(figsize=(8, 6))
156     for y, lab in ((r.T_Tstar, r"$T/T^*$"), (r.p_pstar, r"$p/p^*$"), (r.T0_T0star,
    r"$T_0/T_0^*$"),
157                    (r.p0_p0star, r"$p_0/p_0^*$"), (r.V_Vstar, r"$V/V^*$")):
158         ax.plot(M, y, label=lab)
159     ax.set(xlabel="Mach number", ylabel="Ratio to sonic state", ylim=(0, 3),
160            title="Rayleigh flow (heat addition, constant area)")
161     ax.legend(); ax.grid(True, **grid)
162     _finish(fig, plt, "rayleigh_curves.png", show)
163
164     # 6. Fanno
165     M = np.linspace(0.05, 5, 400)
166     f = cf.fanno_ratios(M, gamma)
167     fig, ax = plt.subplots(figsize=(8, 6))
168     for y, lab in ((f.T_Tstar, r"$T/T^*$"), (f.p_pstar, r"$p/p^*$"), (f.p0_p0star,
    r"$p_0/p_0^*$"),
169                    (f.V_Vstar, r"$V/V^*$"), (f.fLstar_D, r"$fL^*/D$")):
170         ax.plot(M, y, label=lab)
171     ax.set(xlabel="Mach number", ylabel="Ratio to sonic state", ylim=(0, 4),
172            title="Fanno flow (adiabatic, constant area, friction)")
173     ax.legend(); ax.grid(True, **grid)
174     _finish(fig, plt, "fanno_curves.png", show)
175
176     print(f"Saved 6 figures to ./{FIG_DIR}/")
177
178
179 # ---------------------------------------------------------------------------- #
180 # Printing helpers
181 # ---------------------------------------------------------------------------- #
182 def print_summary(M, gamma=1.4):
183     print(f"\n--- Isentropic relations (M = {M:g}, gamma = {gamma:g}) ---")
184     s = cf.isentropic_ratios(M, gamma)
185     print(f"  T0/T = {s.T0_T:.4f}   p0/p = {s.p0_p:.4f}   rho0/rho = {s.rho0_rho:.4f}")
186     if M > 0:
187         print(f"  A/A* = {cf.area_mach_ratio(M, gamma):.4f}")
188     if M >= 1:
189         n = cf.normal_shock(M, gamma)
190         print(f"\n--- Normal shock (M1 = {M:g}) ---")
191         print(f"  M2 = {n.M2:.4f}   p2/p1 = {n.p2_p1:.4f}   rho2/rho1 =
    {n.rho2_rho1:.4f}")
192         print(f"  T2/T1 = {n.T2_T1:.4f}   p02/p01 = {n.p02_p01:.4f}   "
193               f"(s2-s1)/R = {cf.shock_entropy_rise(M, gamma):.4f}")
194         print(f"  Pitot p02/p1 = {cf.pitot_pressure_ratio(M, gamma):.4f}")
195         print(f"\n--- Prandtl-Meyer (M = {M:g}) ---")
196         print(f"  nu = {cf.prandtl_meyer(M, gamma):.4f} deg   (nu_max =
    {cf.prandtl_meyer_max(gamma):.2f} deg)")
197         if M > 1:
198             print(f"  Mach angle mu = {cf.mach_angle_deg(M):.3f} deg   "
199                   f"theta_max (oblique) = {cf.max_deflection_angle(M, gamma)[0]:.3f}
    deg")
200     else:
201         print("\n(Shock and Prandtl-Meyer relations need M >= 1 - skipped.)")
202     if M > 0:
203         r = cf.rayleigh_ratios(M, gamma)
204         f = cf.fanno_ratios(M, gamma)
205         print(f"\n--- Rayleigh flow (M = {M:g}) ---")
206         print(f"  T/T* = {r.T_Tstar:.4f}   p/p* = {r.p_pstar:.4f}   T0/T0* =
    {r.T0_T0star:.4f}   p0/p0* = {r.p0_p0star:.4f}")
207         print(f"\n--- Fanno flow (M = {M:g}) ---")
208         print(f"  T/T* = {f.T_Tstar:.4f}   p/p* = {f.p_pstar:.4f}   p0/p0* =
    {f.p0_p0star:.4f}   fL*/D = {f.fLstar_D:.4f}")
209     print()
210
211
```

```
212 def _ask_float(prompt, default=None):
213     raw = input(prompt).strip()
214     if not raw and default is not None:
215         return default
216     return float(raw)
217 
218 
219 def interactive_menu(gamma=1.4):
220     menu = (
221         "\n" + "=" * 60 + "\nCOMPRESSIBLE FLOW CALCULATOR\n" + "=" * 60 + "\n"
222         "  1  Full summary at one Mach number\n"
223         "  2  Oblique shock (M1 + wedge angle theta)\n"
224         "  3  Prandtl-Meyer expansion (M1 + turn angle)\n"
225         "  4  Isentropic: Mach number from A/A*\n"
226         "  5  Rayleigh duct: exit Mach for T02/T01\n"
227         "  6  Fanno duct: exit Mach for f L/D\n"
228         "  7  Change gamma (currently {g:g})\n"
229         "  q  Quit\n"
230     )
231     while True:
232         print(menu.format(g=gamma))
233         try:
234             choice = input("Choose an option: ").strip().lower()
235             if choice in ("q", "quit", "exit", ""):
236                 break
237             if choice == "1":
238                 print_summary(_ask_float("Mach number: "), gamma)
239             elif choice == "2":
240                 M1 = _ask_float("Upstream Mach number M1: ")
241                 th = _ask_float("Deflection angle theta (deg): ")
242                 branch = input("Solution branch - weak or strong [weak]:
    ").strip().lower() or "weak"
243                 o = cf.oblique_shock(M1, theta_deg=th, weak=(branch != "strong"),
    gamma=gamma)
244                 print(f"\n  beta = {o.beta_deg:.3f} deg   Mn1 = {o.Mn1:.4f}   M2 =
    {o.M2:.4f}")
245                 print(f"  p2/p1 = {o.p2_p1:.4f}   rho2/rho1 = {o.rho2_rho1:.4f}   "
246                       f"T2/T1 = {o.T2_T1:.4f}   p02/p01 = {o.p02_p01:.4f}")
247             elif choice == "3":
248                 M1 = _ask_float("Upstream Mach number M1: ")
249                 turn = _ask_float("Turn angle (deg): ")
250                 e = cf.prandtl_meyer_expansion(M1, turn, gamma)
251                 print(f"\n  M2 = {e.M2:.4f}   nu1 = {e.nu1_deg:.3f}   nu2 =
    {e.nu2_deg:.3f} deg")
252                 print(f"  p2/p1 = {e.p2_p1:.4f}   rho2/rho1 = {e.rho2_rho1:.4f}   T2/T1
    = {e.T2_T1:.4f}")
253                 print(f"  fan between Mach waves {e.mu1_deg:.2f} deg and
    {e.mu2_deg:.2f} deg")
254             elif choice == "4":
255                 A = _ask_float("A/A* (>= 1): ")
256                 print(f"\n  Subsonic   M = {cf.mach_from_area_ratio(A,
    supersonic=False, gamma=gamma):.4f}")
257                 print(f"  Supersonic M = {cf.mach_from_area_ratio(A, supersonic=True,
    gamma=gamma):.4f}")
258             elif choice == "5":
259                 M1 = _ask_float("Inlet Mach number: ")
260                 ratio = _ask_float("Stagnation-temperature ratio T02/T01: ")
261                 print(f"\n  Exit Mach number = {cf.rayleigh_exit(M1, ratio,
    gamma):.4f}")
262             elif choice == "6":
263                 M1 = _ask_float("Inlet Mach number: ")
264                 fld = _ask_float("Friction parameter f L/D (Darcy f): ")
265                 print(f"\n  Exit Mach number = {cf.fanno_exit(M1, fld, gamma):.4f}")
266             elif choice == "7":
267                 new = _ask_float("New gamma (> 1): ")
268                 cf.isentropic_ratios(1.0, new)  # validates
269                 gamma = new
270             else:
271                 print("Unknown option.")
272         except ValueError as exc:
273             print(f"\n  Input error: {exc}")
274         except (EOFError, KeyboardInterrupt):
275             print()
276             break
277     print("Exiting. Figures (if generated) are in ./" + FIG_DIR + "/")
278 
279 
```

```
280 # ---------------------------------------------------------------------------- #
281 def main(argv=None):
282     ap = argparse.ArgumentParser(description="Compressible flow calculator (calorically
    perfect gas).")
283     ap.add_argument("--mach", type=float, help="print a one-shot summary at this Mach
    number")
284     ap.add_argument("--gamma", type=float, default=1.4, help="ratio of specific heats
    (default 1.4)")
285     ap.add_argument("--validate", action="store_true", help="run validation only")
286     ap.add_argument("--plots", action="store_true", help="generate figures only")
287     ap.add_argument("--no-plots", action="store_true", help="skip figure generation")
288     ap.add_argument("--no-interactive", action="store_true", help="skip the interactive
    menu")
289     ap.add_argument("--show", action="store_true", help="display plot windows (blocks
    until closed)")
290     a = ap.parse_args(argv)
291
292     try:
293         cf.isentropic_ratios(1.0, a.gamma)  # validate gamma early
294     except ValueError as exc:
295         ap.error(str(exc))
296
297     if a.validate:
298         return 0 if run_validation() else 1
299     if a.plots:
300         make_plots(a.gamma, a.show)
301         return 0
302     if a.mach is not None:
303         try:
304             print_summary(a.mach, a.gamma)
305         except ValueError as exc:
306             ap.error(str(exc))
307         return 0
308
309     ok = run_validation()
310     if not a.no_plots:
311         make_plots(a.gamma, a.show)
312     if not a.no_interactive:
313         interactive_menu(a.gamma)
314     return 0 if ok else 1
315
316
317 if __name__ == "__main__":
318     sys.exit(main())
```

Appendix F test_compressible_flow.py (library tests)

Complete source of the 108 library tests, grouped by relation family. Reference values are taken from NACA Report 1135 and Anderson [1, 2]; the remaining tests check physical identities, error handling and the near-sonic regression.

```
"""
Test-suite for compressible_flow.py

Reference values (gamma = 1.4) come from the published tables in
NACA Report 1135 and Anderson, "Modern Compressible Flow" (Appendices A-C).
Table values are quoted to 4-5 significant figures, so tolerances are set to the
last quoted digit.
"""

import numpy as np
import pytest

import compressible_flow as cf

G = 1.4


# ---------------------------------------------------------------- isentropic
class TestIsentropic:
    @pytest.mark.parametrize(
        "M, T0_T, p0_p, rho0_rho, A_Astar",
        [
            (0.5, 1.0500, 1.1862, 1.1297, 1.3398),
            (1.0, 1.2000, 1.8929, 1.5774, 1.0000),
            (2.0, 1.8000, 7.8244, 4.3469, 1.6875),
            (3.0, 2.8000, 36.733, 13.119, 4.2346),  # rho0/rho = 2.8**2.5
        ],
    )
    def test_table_values(self, M, T0_T, p0_p, rho0_rho, A_Astar):
        s = cf.isentropic_ratios(M)
        assert s.T0_T == pytest.approx(T0_T, abs=5e-5)
        assert s.p0_p == pytest.approx(p0_p, rel=2e-4)
        assert s.rho0_rho == pytest.approx(rho0_rho, rel=2e-4)
        assert cf.area_mach_ratio(M) == pytest.approx(A_Astar, rel=2e-4)

    def test_ideal_gas_consistency(self):
        # p0/p = (rho0/rho) * (T0/T) for an ideal gas
        s = cf.isentropic_ratios(1.7, 1.3)
        assert s.p0_p == pytest.approx(s.rho0_rho * s.T0_T)

    def test_zero_mach_is_stagnation(self):
        assert tuple(cf.isentropic_ratios(0.0)) == (1.0, 1.0, 1.0)

    @pytest.mark.parametrize("M", [0.3, 0.9, 1.5, 4.0, 12.0])
    def test_area_inverse_roundtrip(self, M):
        A = cf.area_mach_ratio(M)
        assert cf.mach_from_area_ratio(A, supersonic=M > 1) == pytest.approx(M, rel=1e-
9)

    def test_area_inverse_sonic(self):
        assert cf.mach_from_area_ratio(1.0) == 1.0

    def test_vectorised(self):
        M = np.array([0.5, 1.0, 2.0])
        out = cf.isentropic_ratios(M)
        assert out.p0_p.shape == (3,)
        assert out.p0_p[2] == pytest.approx(7.8244, rel=2e-4)

    @pytest.mark.parametrize("bad", [-1.0, np.nan, np.inf])
    def test_rejects_bad_mach(self, bad):
        with pytest.raises(ValueError):
            cf.isentropic_ratios(bad)

    @pytest.mark.parametrize("g", [1.0, 0.9, -1.4, np.nan])
    def test_rejects_bad_gamma(self, g):
        with pytest.raises(ValueError):
```

```
 66              cf.isentropic_ratios(2.0, g)
 67
 68      def test_area_ratio_zero_mach_rejected(self):
 69          # the original code raised ZeroDivisionError here
 70          with pytest.raises(ValueError):
 71              cf.area_mach_ratio(0.0)
 72
 73      def test_area_inverse_rejects_below_one(self):
 74          with pytest.raises(ValueError):
 75              cf.mach_from_area_ratio(0.9)
 76
 77
 78  # ------------------------------------------------------------------ normal shock
 79  class TestNormalShock:
 80      @pytest.mark.parametrize(
 81          "M1, M2, p, rho, T, p0",
 82          [
 83              (2.0, 0.5774, 4.5000, 2.6667, 1.6875, 0.7209),
 84              (3.0, 0.4752, 10.333, 3.8571, 2.6790, 0.3283),
 85              (5.0, 0.4152, 29.000, 5.0000, 5.8000, 0.06172),
 86          ],
 87      )
 88      def test_table_values(self, M1, M2, p, rho, T, p0):
 89          s = cf.normal_shock(M1)
 90          assert s.M2 == pytest.approx(M2, abs=1e-4)
 91          assert s.p2_p1 == pytest.approx(p, rel=1e-4)
 92          assert s.rho2_rho1 == pytest.approx(rho, rel=1e-4)
 93          assert s.T2_T1 == pytest.approx(T, rel=1e-4)
 94          assert s.p02_p01 == pytest.approx(p0, rel=2e-3)
 95
 96      def test_sonic_limit_is_no_shock(self):
 97          assert all(v == pytest.approx(1.0) for v in cf.normal_shock(1.0))
 98
 99      def test_subsonic_rejected(self):
100          with pytest.raises(ValueError):
101              cf.normal_shock(0.8)
102
103      def test_downstream_subsonic_and_loses_pressure(self):
104          for M1 in (1.1, 1.5, 2.5, 10.0):
105              s = cf.normal_shock(M1)
106              assert s.M2 < 1.0
107              assert 0.0 < s.p02_p01 < 1.0
108
109      def test_strong_shock_density_limit(self):
110          # rho2/rho1 -> (g+1)/(g-1) = 6 as M1 -> infinity
111          assert cf.normal_shock(1e3).rho2_rho1 == pytest.approx(6.0, rel=1e-3)
112
113      def test_entropy_rise_positive(self):
114          assert cf.shock_entropy_rise(2.0) == pytest.approx(-np.log(0.7209), rel=2e-3)
115          assert cf.shock_entropy_rise(1.0) == pytest.approx(0.0, abs=1e-12)
116
117      def test_pitot_ratio_table(self):
118          assert cf.pitot_pressure_ratio(2.0) == pytest.approx(5.6404, rel=1e-4)
119
120      def test_pitot_consistent_with_shock_plus_isentropic(self):
121          M1 = 2.7
122          s = cf.normal_shock(M1)
123          p01_p1 = cf.isentropic_ratios(M1).p0_p
124          assert cf.pitot_pressure_ratio(M1) == pytest.approx(s.p02_p01 * p01_p1)
125
126      def test_inverse_relations(self):
127          s = cf.normal_shock(3.3)
128          assert cf.mach1_from_pressure_ratio(s.p2_p1) == pytest.approx(3.3)
129          assert cf.mach1_from_downstream_mach(s.M2) == pytest.approx(3.3)
130
131      def test_inverse_bounds(self):
132          with pytest.raises(ValueError):
133              cf.mach1_from_pressure_ratio(0.5)
134          with pytest.raises(ValueError):
135              cf.mach1_from_downstream_mach(0.2)  # below sqrt((g-1)/2g) = 0.378
136
137      def test_vectorised_shock(self):
138          # the original implementation failed on array input
139          s = cf.normal_shock(np.array([2.0, 3.0]))
140          assert s.p2_p1[1] == pytest.approx(10.333, rel=1e-4)
141
142
```

```
143 # ------------------------------------------------------------------ oblique shock
144 class TestObliqueShock:
145     def test_table_M2_theta10(self):
146         s = cf.oblique_shock(2.0, theta_deg=10.0)
147         assert s.beta_deg == pytest.approx(39.31, abs=0.01)
148         assert s.M2 == pytest.approx(1.641, abs=2e-3)
149         assert s.p2_p1 == pytest.approx(1.707, abs=2e-3)
150 
151     def test_textbook_example_M2_theta20(self):
152         # Anderson Example 4.1-style case: M1 = 2, theta = 20 deg -> beta ~ 53.4 deg
153         s = cf.oblique_shock(2.0, theta_deg=20.0)
154         assert s.beta_deg == pytest.approx(53.4, abs=0.1)
155         assert s.Mn1 == pytest.approx(1.60, abs=0.01)
156         assert s.M2 == pytest.approx(1.21, abs=0.01)
157         # The textbook rounds Mn1 to 1.60 -> p2/p1 = 2.82; the exact Mn1 = 1.607 gives
    2.843.
158         assert s.p2_p1 == pytest.approx(2.82, abs=0.03)
159         assert s.p2_p1 == pytest.approx(1 + 2.8 / 2.4 * (s.Mn1**2 - 1), rel=1e-12)
160 
161     def test_max_deflection_values(self):
162         theta_max, _ = cf.max_deflection_angle(2.0)
163         assert theta_max == pytest.approx(22.97, abs=0.02)
164         assert cf.max_deflection_angle(1e4)[0] == pytest.approx(45.58, abs=0.05)
165 
166     def test_detached_shock_rejected(self):
167         with pytest.raises(ValueError, match="detach"):
168             cf.oblique_shock(2.0, theta_deg=25.0)
169 
170     def test_weak_vs_strong_branches(self):
171         weak = cf.oblique_shock(3.0, theta_deg=15.0, weak=True)
172         strong = cf.oblique_shock(3.0, theta_deg=15.0, weak=False)
173         assert weak.beta_deg < strong.beta_deg
174         assert weak.M2 > 1.0       # weak solution: supersonic downstream
175         assert strong.M2 < 1.0     # strong solution: subsonic downstream
176         assert strong.p2_p1 > weak.p2_p1
177         # both must reproduce the same deflection angle
178         for s in (weak, strong):
179             assert cf.theta_from_beta(3.0, s.beta_deg) == pytest.approx(15.0, abs=1e-9)
180 
181     def test_beta_input_roundtrip(self):
182         s1 = cf.oblique_shock(2.5, theta_deg=12.0)
183         s2 = cf.oblique_shock(2.5, beta_deg=s1.beta_deg)
184         assert s2.theta_deg == pytest.approx(12.0, abs=1e-9)
185         assert s2.M2 == pytest.approx(s1.M2)
186 
187     def test_normal_shock_limit(self):
188         # beta = 90 deg must reduce to the normal shock
189         o = cf.oblique_shock(2.0, beta_deg=90.0)
190         n = cf.normal_shock(2.0)
191         assert o.theta_deg == pytest.approx(0.0, abs=1e-9)
192         assert o.p2_p1 == pytest.approx(n.p2_p1)
193         assert o.M2 == pytest.approx(n.M2)
194 
195     def test_zero_deflection_is_mach_wave(self):
196         s = cf.oblique_shock(2.0, theta_deg=0.0)
197         assert s.beta_deg == pytest.approx(30.0)  # arcsin(1/2)
198         assert s.p2_p1 == pytest.approx(1.0)
199 
200     def test_requires_exactly_one_angle(self):
201         with pytest.raises(ValueError):
202             cf.oblique_shock(2.0)
203         with pytest.raises(ValueError):
204             cf.oblique_shock(2.0, theta_deg=10.0, beta_deg=40.0)
205 
206     def test_beta_below_mach_angle_rejected(self):
207         with pytest.raises(ValueError):
208             cf.oblique_shock(2.0, beta_deg=20.0)
209 
210     def test_tangential_velocity_conserved(self):
211         # V_t is continuous across an oblique shock: check with temperatures (a ~
    sqrt(T))
212         M1 = 3.0
213         s = cf.oblique_shock(M1, theta_deg=14.0)
214         b, th = np.radians(s.beta_deg), np.radians(s.theta_deg)
215         a2_a1 = np.sqrt(s.T2_T1)
216         Vt1 = M1 * np.cos(b)
217         Vt2 = s.M2 * a2_a1 * np.cos(b - th)
```

```
218         assert Vt2 == pytest.approx(Vt1, rel=1e-9)
219
220     def test_mach_angle(self):
221         assert cf.mach_angle_deg(2.0) == pytest.approx(30.0)
222         assert cf.mach_angle_deg(1.0) == pytest.approx(90.0)
223
224
225 # ------------------------------------------------------------------ Prandtl-Meyer
226 class TestPrandtlMeyer:
227     @pytest.mark.parametrize("M, nu", [(1.0, 0.0), (2.0, 26.380), (3.0, 49.757), (5.0,
    76.920)])
228     def test_table_values(self, M, nu):
229         assert cf.prandtl_meyer(M) == pytest.approx(nu, abs=2e-3)
230
231     def test_limit(self):
232         assert cf.prandtl_meyer_max() == pytest.approx(130.454, abs=1e-3)
233         assert cf.prandtl_meyer(1e6) == pytest.approx(130.454, abs=1e-2)
234
235     def test_monotonic(self):
236         nu = cf.prandtl_meyer(np.linspace(1.0, 10.0, 200))
237         assert np.all(np.diff(nu) > 0)
238
239     @pytest.mark.parametrize("M", [1.0001, 1.5, 2.0, 5.0, 20.0])
240     def test_inverse_roundtrip(self, M):
241         assert cf.mach_from_pm_angle(cf.prandtl_meyer(M)) == pytest.approx(M, rel=1e-8)
242
243     def test_inverse_zero_and_unreachable(self):
244         assert cf.mach_from_pm_angle(0.0) == 1.0   # failed in the original code
245         assert cf.mach_from_pm_angle(130.0) > 100  # close to the limit, still solvable
246         with pytest.raises(ValueError):
247             cf.mach_from_pm_angle(131.0)
248         with pytest.raises(ValueError):
249             cf.mach_from_pm_angle(-1.0)
250
251     def test_subsonic_rejected(self):
252         with pytest.raises(ValueError):
253             cf.prandtl_meyer(0.9)
254
255     def test_expansion_fan(self):
256         e = cf.prandtl_meyer_expansion(2.0, 10.0)
257         assert e.nu2_deg == pytest.approx(36.380, abs=2e-3)
258         assert e.M2 == pytest.approx(2.386, abs=2e-3)
259         assert e.p2_p1 < 1.0 and e.T2_T1 < 1.0 and e.rho2_rho1 < 1.0
260         assert e.mu2_deg < e.mu1_deg
261         # isentropic: stagnation conditions unchanged
262         p0 = cf.isentropic_ratios(2.0).p0_p / cf.isentropic_ratios(e.M2).p0_p
263         assert e.p2_p1 == pytest.approx(p0)
264
265     def test_zero_turn_is_identity(self):
266         e = cf.prandtl_meyer_expansion(3.0, 0.0)
267         assert e.M2 == pytest.approx(3.0)
268         assert e.p2_p1 == pytest.approx(1.0)
269
270
271 # ------------------------------------------------------------------ Rayleigh
272 class TestRayleigh:
273     def test_sonic_reference(self):
274         assert all(v == pytest.approx(1.0) for v in cf.rayleigh_ratios(1.0))
275
276     @pytest.mark.parametrize(
277         "M, T, p, T0, p0",
278         [
279             (0.5, 0.7901, 1.7778, 0.6914, 1.1141),
280             (2.0, 0.5289, 0.3636, 0.7934, 1.5031),
281         ],
282     )
283     def test_table_values(self, M, T, p, T0, p0):
284         r = cf.rayleigh_ratios(M)
285         assert r.T_Tstar == pytest.approx(T, abs=1e-4)
286         assert r.p_pstar == pytest.approx(p, abs=1e-4)
287         assert r.T0_T0star == pytest.approx(T0, abs=1e-4)
288         assert r.p0_p0star == pytest.approx(p0, abs=1e-4)
289
290     def test_velocity_density_reciprocal(self):
291         r = cf.rayleigh_ratios(2.0)
292         assert r.rho_rhostar * r.V_Vstar == pytest.approx(1.0)
293         assert r.V_Vstar == pytest.approx(1.4545, abs=1e-4)
```

```
294
295     @pytest.mark.parametrize("M", [0.2, 0.7, 1.8, 6.0])
296     def test_inverse_roundtrip(self, M):
297         r = cf.rayleigh_ratios(M).T0_T0star
298         assert cf.mach_from_rayleigh_T0_ratio(r, supersonic=M > 1) == pytest.approx(M,
    rel=1e-8)
299
300     def test_supersonic_floor(self):
301         with pytest.raises(ValueError):
302             cf.mach_from_rayleigh_T0_ratio(0.40, supersonic=True)  # floor ~0.4898
303
304     def test_heating_accelerates_subsonic_flow(self):
305         M2 = cf.rayleigh_exit(0.3, 2.0)
306         assert 0.3 < M2 < 1.0
307
308     def test_heating_decelerates_supersonic_flow(self):
309         M2 = cf.rayleigh_exit(3.0, 1.5)
310         assert 1.0 < M2 < 3.0
311
312     def test_cooling_reverses_trend(self):
313         assert cf.rayleigh_exit(0.5, 0.8) < 0.5
314
315     def test_thermal_choking(self):
316         ratio_choke = 1.0 / cf.rayleigh_ratios(0.5).T0_T0star
317         assert cf.rayleigh_exit(0.5, ratio_choke * 0.999999) == pytest.approx(1.0,
    abs=1e-2)
318         with pytest.raises(ValueError, match="choked"):
319             cf.rayleigh_exit(0.5, ratio_choke * 1.01)
320
321
322 # ------------------------------------------------------------------------ Fanno
323 class TestFanno:
324     def test_sonic_reference(self):
325         f = cf.fanno_ratios(1.0)
326         assert all(v == pytest.approx(1.0) for v in f[:5])
327         assert f.fLstar_D == pytest.approx(0.0, abs=1e-12)
328
329     @pytest.mark.parametrize(
330         "M, T, p, rho, V, p0, fL",
331         [
332             (0.5, 1.1429, 2.1381, 1.8708, 0.5345, 1.3398, 1.0691),
333             (2.0, 0.6667, 0.4082, 0.6124, 1.6330, 1.6875, 0.3050),
334         ],
335     )
336     def test_table_values(self, M, T, p, rho, V, p0, fL):
337         f = cf.fanno_ratios(M)
338         assert f.T_Tstar == pytest.approx(T, abs=1e-4)
339         assert f.p_pstar == pytest.approx(p, abs=1e-4)
340         assert f.rho_rhostar == pytest.approx(rho, abs=1e-4)
341         assert f.V_Vstar == pytest.approx(V, abs=1e-4)
342         assert f.p0_p0star == pytest.approx(p0, abs=1e-4)
343         assert f.fLstar_D == pytest.approx(fL, abs=1e-4)
344
345     def test_p0_ratio_equals_area_ratio(self):
346         # For Fanno flow p0/p0* is identical to the isentropic A/A*
347         for M in (0.3, 0.8, 1.5, 3.0):
348             assert cf.fanno_ratios(M).p0_p0star == pytest.approx(cf.area_mach_ratio(M))
349
350     def test_ideal_gas_consistency(self):
351         # p = rho R T  =>  p/p* = (rho/rho*)(T/T*)
352         f = cf.fanno_ratios(0.6)
353         assert f.p_pstar == pytest.approx(f.rho_rhostar * f.T_Tstar)
354
355     def test_supersonic_limit(self):
356         assert cf.fanno_parameter(1e6) == pytest.approx(0.8215, abs=1e-3)
357
358     @pytest.mark.parametrize("M", [0.1, 0.5, 0.9, 1.5, 3.0, 10.0])
359     def test_inverse_roundtrip(self, M):
360         p = cf.fanno_parameter(M)
361         assert cf.mach_from_fanno_parameter(p, supersonic=M > 1) == pytest.approx(M,
    rel=1e-7)
362
363     def test_inverse_rejects_beyond_limit(self):
364         with pytest.raises(ValueError):
365             cf.mach_from_fanno_parameter(0.9, supersonic=True)
366
367     def test_subsonic_friction_accelerates_to_sonic(self):
```

```
368         M2 = cf.fanno_exit(0.5, 0.5)
369         assert 0.5 < M2 < 1.0
370         assert cf.fanno_exit(0.5, cf.fanno_parameter(0.5)) == pytest.approx(1.0)
371
372     def test_supersonic_friction_decelerates(self):
373         M2 = cf.fanno_exit(3.0, 0.2)
374         assert 1.0 < M2 < 3.0
375
376     def test_choked_duct(self):
377         with pytest.raises(ValueError, match="Choked"):
378             cf.fanno_exit(0.5, 1.2)
379
380     def test_exit_additivity(self):
381         # marching in two legs must equal one long leg
382         M_mid = cf.fanno_exit(0.3, 0.4)
383         M_end_two = cf.fanno_exit(M_mid, 0.5)
384         M_end_one = cf.fanno_exit(0.3, 0.9)
385         assert M_end_two == pytest.approx(M_end_one, rel=1e-8)
386
387
388 # ------------------------------------------------ remaining input-validation paths
389 class TestInputValidation:
390     def test_oblique_negative_theta(self):
391         with pytest.raises(ValueError):
392             cf.oblique_shock(2.0, theta_deg=-5.0)
393
394     def test_negative_expansion_turn(self):
395         with pytest.raises(ValueError):
396             cf.prandtl_meyer_expansion(2.0, -1.0)
397
398     def test_rayleigh_ratio_domain(self):
399         with pytest.raises(ValueError):
400             cf.mach_from_rayleigh_T0_ratio(1.5, supersonic=False)
401         with pytest.raises(ValueError):
402             cf.rayleigh_exit(0.5, -1.0)
403
404     def test_fanno_negative_parameter(self):
405         with pytest.raises(ValueError):
406             cf.mach_from_fanno_parameter(-0.1, supersonic=False)
407         with pytest.raises(ValueError):
408             cf.fanno_exit(0.5, -0.1)
409
410     def test_pm_inverse_extremely_close_to_limit(self):
411         # bracket expansion must either succeed or fail with a clear ValueError, never
    hang
412         try:
413             M = cf.mach_from_pm_angle(cf.prandtl_meyer_max() - 1e-9)
414             assert M > 1e3
415         except ValueError:
416             pass
417
418
419 # ------------------------------------------- regression: near-sonic inverse accuracy
420 class TestNearSonicRegression:
421     """np.isclose(..., atol=1e-12) used the default rtol=1e-5 and returned M=1.0 for
422     M = 1.001 (error 1e-3). Quantities such as A/A* - 1 are quadratic in (M - 1)."""
423
424     @pytest.mark.parametrize("M", [1.0005, 1.001, 1.01, 0.999, 0.99])
425     def test_area_inverse_near_sonic(self, M):
426         A = cf.area_mach_ratio(M)
427         assert cf.mach_from_area_ratio(A, supersonic=M > 1) == pytest.approx(M, rel=1e-
    6)
428
429     @pytest.mark.parametrize("M", [1.0005, 1.001, 1.01, 0.999, 0.99])
430     def test_rayleigh_inverse_near_sonic(self, M):
431         r = cf.rayleigh_ratios(M).T0_T0star
432         assert cf.mach_from_rayleigh_T0_ratio(r, supersonic=M > 1) == pytest.approx(M,
    rel=1e-6)
433
434     def test_full_sweep_supersonic_area_inverse(self):
435         worst = max(
436             abs(cf.mach_from_area_ratio(cf.area_mach_ratio(m), True) - m) / m
437             for m in np.linspace(1.001, 30, 120)
438         )
439         assert worst < 1e-8
```

Appendix G test_main.py (command-line tests)

Complete source of the 8 smoke tests for the command-line front end.

```
"""Smoke tests for the command-line front end (main.py)."""

import os

import pytest

import main as cli


def test_validation_passes(capsys):
    assert cli.run_validation(verbose=True) is True
    assert "16/16 checks passed" in capsys.readouterr().out


def test_one_shot_summary(capsys):
    assert cli.main(["--mach", "2.0", "--no-plots"]) == 0
    out = capsys.readouterr().out
    assert "p2/p1 = 4.5000" in out and "nu = 26.3798" in out


def test_subsonic_summary_skips_shock(capsys):
    cli.main(["--mach", "0.5"])
    assert "skipped" in capsys.readouterr().out


@pytest.mark.parametrize("argv", [["--gamma", "0.8", "--mach", "2"], ["--mach", "-1"]])
def test_bad_arguments_exit_cleanly(argv):
    with pytest.raises(SystemExit) as e:
        cli.main(argv)
    assert e.value.code == 2


def test_plots_written(tmp_path, monkeypatch):
    monkeypatch.setattr(cli, "FIG_DIR", str(tmp_path))
    assert cli.main(["--plots"]) == 0
    names = set(os.listdir(tmp_path))
    assert {"isentropic_curves.png", "normal_shock_curves.png",
"prandtl_meyer_curve.png",
            "theta_beta_M_chart.png", "rayleigh_curves.png", "fanno_curves.png"} <=
names


def test_interactive_menu_handles_errors(monkeypatch, capsys):
    # oblique shock past detachment -> message, not a crash; then quit
    inputs = iter(["2", "2", "25", "weak", "4", "0.5", "q"])
    monkeypatch.setattr("builtins.input", lambda *_: next(inputs))
    cli.interactive_menu()
    out = capsys.readouterr().out
    assert "detaches" in out and "A/A* must be >= 1" in out


def test_interactive_menu_survives_eof(monkeypatch):
    def boom(*_):
        raise EOFError
    monkeypatch.setattr("builtins.input", boom)
    cli.interactive_menu()  # must return, not raise
```